Kubernetes Networking Deep-Dive

A Beginner-to-Advanced Technical Teaching Guide

Repository: `1629lyk/k8s-networking-project`

Technical Reference Document

Stack: Ubuntu 22.04 WSL2 · kind v0.31.0 · Calico v3.27.0 · etcd v3.5.0

Target Reader: CCNA-certified engineer, new to Kubernetes

April 2026

Abstract

This document is a structured, beginner-to-advanced teaching guide for the `1629lyk/k8s-networking-pro` Kubernetes lab. It targets engineers who hold a CCNA certification and understand campus networking (IP addressing, VLANs, STP, OSPF, NAT, ACLs) but who have not yet worked deeply with Kubernetes, container networking, or distributed systems.

The guide bridges the conceptual gap from traditional networking to cloud-native infrastructure, covering every major technology in the project: WSL2, Docker, kind, kubectl, Helm, Calico CNI, kube-proxy, CoreDNS, NetworkPolicy, Linux network namespaces, tcpdump, conntrack, nsenter, Goldpinger, etcd, and Raft consensus. It includes end-to-end architecture diagrams, packet-level traffic walkthroughs, a step-by-step verification lab guide, a critical repository evaluation, interview preparation, a study roadmap, and a categorised reference library of 50+ authoritative sources.

Contents

1 Executive Overview **1**
1.1 What This Project Is . . . 1
1.2 What Problem It Solves . . . 1
1.3 Why This Lab Exists . . . 2
1.4 Where This Fits in Real-World Engineering . . . 2
1.5 Why This Project Is Beyond CCNA . . . 2

2 Prerequisite Bridge: From CCNA to This Project **3**
2.1 What CCNA Prepares You For . . . 3
2.2 The Core Conceptual Shift . . . 4
2.3 How Container Networking Differs from Campus Networking . . . 5
2.3.1 Why Pods Need Their Own IP Addresses . . . 5
2.4 Why Linux Networking Matters Here . . . 5
2.5 Why Distributed Systems Concepts Matter . . . 6

3 Concept-by-Concept Teaching Path **7**
3.1 WSL2 — Windows Subsystem for Linux 2 . . . 7
3.1.1 Plain-English Meaning . . . 7
3.1.2 Technical Meaning . . . 7
3.1.3 The WSL2 kubectl Problem . . . 7
3.1.4 Verification . . . 7
3.1.5 Common Mistakes . . . 7
3.2 Docker — Relevant Concepts for This Lab . . . 8
3.2.1 Bridge Networks . . . 8
3.2.2 Key Docker Commands in This Project . . . 8
3.3 kind — Kubernetes IN Docker . . . 8
3.3.1 What kind Creates . . . 8
3.3.2 The Cluster Config . . . 8
3.4 kubectl — Kubernetes API Client . . . 9
3.5 Helm — Kubernetes Package Manager . . . 9
3.6 Kubernetes Architecture . . . 10
3.6.1 Control Plane vs Data Plane . . . 10
3.6.2 Architecture Diagram . . . 10
3.7 Workload Objects: Pod, Deployment, StatefulSet, DaemonSet . . . 10
3.8 Services, ClusterIP, and Virtual IPs . . . 11
3.8.1 The Problem . . . 11
3.8.2 What a ClusterIP Is . . . 11
3.9 kube-proxy and iptables DNAT . . . 11
3.10 CoreDNS and Kubernetes DNS . . . 12
3.11 CNI — Container Network Interface Specification . . . 12
3.12 Calico CNI . . . 12
3.12.1 IPAM: Per-Node /26 Blocks . . . 12
3.12.2 veth Pairs . . . 12
3.12.3 Cross-Node Routing (VXLANCrossSubnet) . . . 12
3.12.4 Calico and NetworkPolicy . . . 13
3.13 Linux Network Namespaces and veth Pairs . . . 13

3.13.1 Network Namespaces . . . 13
3.13.2 veth Pairs . . . 13
3.14 nsenter — Entering Namespaces from the Host . . . 13
3.15 tcpdump — Packet Capture . . . 13
3.16 conntrack — Connection State Tracking . . . 14
3.17 NetworkPolicy and Zero-Trust Segmentation . . . 14
3.17.1 How NetworkPolicy Works . . . 14
3.17.2 3-Tier Architecture in Phase 3 . . . 14
3.18 Goldpinger . . . 15
3.19 etcd . . . 15
3.20 Raft Consensus Algorithm . . . 15
3.20.1 Key Terms . . . 15
3.20.2 Election Timeline (from Phase 7 Logs) . . . 15
3.20.3 Quorum Table . . . 16
3.21 Comparison Tables . . . 16

4 Repository Walkthrough 17
4.1 Repository Structure . . . 17
4.2 README.md . . . 18
4.3 phase1-environment-setup.md . . . 18
4.3.1 Key Steps and Their Purpose . . . 18
4.3.2 Fragility Notes . . . 19
4.4 phase2-calico-cni.md . . . 19
4.4.1 Step-by-Step Analysis . . . 19
4.4.2 What This Phase Proves . . . 20
4.5 phase3-networkpolicy.md . . . 20
4.5.1 Step-by-Step Analysis . . . 20
4.5.2 Fragility Notes . . . 21
4.6 phase4-services-dns.md . . . 21
4.6.1 Step-by-Step Analysis . . . 21
4.7 phase5-packet-capture.md . . . 22
4.7.1 Step-by-Step Analysis . . . 22
4.7.2 Quick Reference: nsenter Patterns . . . 22
4.8 phase6-observability.md . . . 23
4.8.1 eBPF vs iptables Section . . . 23
4.8.2 Goldpinger Deployment Analysis . . . 23
4.9 phase7-raft-failures.md . . . 23
4.9.1 etcd StatefulSet Analysis . . . 23
4.9.2 Failure Mode 1 — Leader Node Loss . . . 24
4.9.3 Failure Mode 2 — DNS Exclusion of NotReady Pods . . . 24
4.9.4 Failure Mode 3 — Bootstrap Conflict . . . 24
4.10 issues.md . . . 24
4.10.1 Issue 4 — EOF Piping Limitation . . . 25

5 End-to-End Architecture and Workflow 26
5.1 The Full Stack in This Lab . . . 26
5.2 Phase-by-Phase State Transitions . . . 26
5.2.1 After Phase 1: Cluster with No CNI . . . 26

5.2.2 After Phase 2: Calico Installed . . . 26
5.2.3 After Phase 3: Zero-Trust Segmentation . . . 26
5.2.4 After Phase 4: Services and DNS . . . 27
5.2.5 After Phase 5: Observability . . . 27
5.2.6 After Phase 6: Goldpinger . . . 27
5.2.7 After Phase 7: etcd Raft . . . 27
5.3 Why Nodes Are NotReady Before CNI . . . 27
5.4 How Pod IP Addresses Are Assigned . . . 28
5.5 How Pod-to-Pod Communication Works . . . 28
5.5.1 Same Node . . . 28
5.5.2 Cross-Node . . . 29
5.6 How Services Work . . . 29
5.7 How DNS Works . . . 29
5.8 How NetworkPolicy Changes Traffic . . . 30
5.9 How etcd and Raft Relate to Cluster Behaviour . . . 30

6 End-to-End Packet and Traffic Walkthroughs 30
6.1 kubectl / API Access Path in This WSL2 + kind Setup . . . 30
6.2 Pod on Node A to Pod on Node A (Same-Node) . . . 31
6.3 Pod on Node A to Pod on Node B (Cross-Node) . . . 31
6.4 Pod to Service ClusterIP . . . 32
6.5 Pod DNS Query to CoreDNS . . . 33
6.6 Packet Drop Due to NetworkPolicy . . . 34
6.7 Cross-Node Health Check via Goldpinger . . . 34
6.8 etcd Leader Failure and Raft Recovery . . . 35
6.9 DNS Failure Triage Decision Tree . . . 36

7 Control Plane vs Data Plane Deep Dive 36
7.1 Separation of Concerns Table . . . 37
7.2 What kind Does (and Doesn't Do) . . . 37
7.3 What Docker Does . . . 38
7.4 What the Kubernetes Control Plane Does . . . 38
7.5 What kubelet Does . . . 38
7.6 What kube-proxy Does . . . 39
7.7 What Calico Does . . . 39
7.8 What the Linux Kernel Networking Does . . . 39
7.9 What CoreDNS Does . . . 40
7.10 What etcd Stores . . . 40
7.11 What Raft Governs . . . 40
7.12 Why Packet Captures and conntrack Are Useful for Debugging . . . 40

8 Verification and Troubleshooting Lab Guide 41
8.1 Phase 1 Verification . . . 41
8.1.1 WSL2 and Docker . . . 41
8.1.2 Tool Versions . . . 41
8.1.3 Cluster Creation . . . 42
8.2 Phase 2 Verification . . . 42
8.2.1 Calico Installation . . . 42
8.2.2 Node Readiness . . . 42

8.2.3 Routing Table . . . 42
8.2.4 Cross-Node Ping . . . 43
8.2.5 veth Interfaces . . . 43
8.3 Phase 3 Verification . . . 43
8.3.1 Default-Deny Working . . . 43
8.3.2 3-Tier Policy Matrix . . . 43
8.4 Phase 4 Verification . . . 43
8.4.1 Service ClusterIP . . . 43
8.4.2 Endpoints Populated . . . 44
8.4.3 iptables DNAT Rules . . . 44
8.4.4 DNS Resolution . . . 44
8.4.5 DNS Failure Signature Test . . . 44
8.5 Phase 5 Verification . . . 44
8.5.1 nsenter Works . . . 44
8.5.2 DNAT Visible in tcpdump . . . 45
8.5.3 conntrack DNAT Pairs . . . 45
8.6 Phase 6 Verification . . . 45
8.6.1 Goldpinger Pods Running . . . 45
8.6.2 Cross-Node Health . . . 45
8.7 Phase 7 Verification . . . 45
8.7.1 etcd Cluster Healthy . . . 45
8.7.2 Raft Write and Read . . . 46
8.7.3 Failure Mode 1 — Leader Loss . . . 46
8.8 Quick Diagnostics Cheat Sheet . . . 46

9 Weaknesses, Oddities, and Cleanup Opportunities 47
9.1 WSL2-Only Assumptions . . . 47
9.2 Environment-Specific Workarounds . . . 48
9.3 Fragile Steps . . . 48
9.4 Manual Steps That Could Be Automated . . . 48
9.5 Places Where Docs May Be Incomplete or Misleading . . . 49
9.6 What Could Be Improved for Professionalism . . . 49
9.7 What Is Excellent . . . 49

10 Interview-Ready Understanding 50
10.1 Elevator-Pitch Levels . . . 50
10.1.1 30-Second Version (Recruiter Screen) . . . 50
10.1.2 2-Minute Version (Hiring Manager) . . . 50
10.1.3 Deep Technical Version (Senior/Staff Engineer) . . . 52
10.2 Topic-by-Topic Deep Technical Answers . . . 52
10.2.1 Calico CNI and Pod Routing . . . 52

11 Learning Roadmap 52
11.1 Tier 1 — Master Now (Weeks 1–4) . . . 52
11.1.1 How to Study Tier 1 . . . 54
11.2 Tier 2 — Study Soon (Months 1–3) . . . 54
11.2.1 Tier 2 Lab Extensions . . . 56
11.3 Tier 3 — Revisit Later (Months 3–6+) . . . 56
11.4 What to Ignore Initially . . . 57

11.5 Lab Practice Methodology 58
11.5.1 The Three-Phase Practice Cycle 58
11.5.2 Suggested Additional Lab Exercises 58
11.6 Certification and Career Path Alignment 59
11.7 Estimated Time Investment 60

12 Reference Library **60**
12.1 Kubernetes Fundamentals 60
12.2 kind — Kubernetes IN Docker 61
12.3 kubectl 61
12.4 Helm 61
12.5 CNI Basics 62
12.6 Calico 62
12.7 Kubernetes Services and kube-proxy 62
12.8 CoreDNS and Kubernetes DNS 63
12.9 NetworkPolicy 63
12.10 Linux Namespaces, veth, iptables, and conntrack 64
12.11 tcpdump and Packet Capture in Containerised Environments 64
12.12 etcd 65
12.13 Raft Consensus Algorithm 65
12.14 Goldpinger 66
12.15 WSL2 Networking Caveats 66
12.16 eBPF and Cilium (Next-Generation Networking) 66
12.17 Quick Reference: Which Source for Which Question 67

1 Executive Overview

1.1 What This Project Is

The `1629lyk/k8s-networking-project` is a hands-on, phase-by-phase Kubernetes networking laboratory built on a local Ubuntu 22.04 WSL2 machine. It uses **kind** (Kubernetes IN Docker) to simulate a three-node Kubernetes cluster entirely inside Docker containers, then systematically layers every major networking component of a production Kubernetes cluster on top of that foundation.

The project is divided into seven sequential phases:

Phase 1. Environment Setup — Installing Docker, kind, kubectl, Helm, and creating the 3-node cluster.

Phase 2. Calico CNI — Installing a production-grade Container Network Interface (CNI) plugin and verifying pod IP assignment and cross-node routing.

Phase 3. NetworkPolicy — Building a zero-trust, 3-tier application network with label-based pod isolation.

Phase 4. Services & DNS — Deploying ClusterIP Services, tracing kube-proxy iptables DNAT rules, and diagnosing DNS failure modes.

Phase 5. Packet Capture — Live traffic analysis using `tcpdump` and `conntrack` via `nsenter`, without any tools installed on the Kubernetes nodes.

Phase 6. Observability — Deploying Goldpinger as a DaemonSet for continuous cross-node connectivity health monitoring.

Phase 7. Raft Failures — Deploying a 3-node etcd Raft cluster inside the lab cluster and deliberately causing three distinct failure modes with documented recovery procedures.

1.2 What Problem It Solves

Enterprise networking teams increasingly need engineers who can reason about Kubernetes traffic paths as fluently as they reason about OSPF routes or iptables ACLs. The problem is that *most Kubernetes tutorials stop at "kubectl apply" and never show you what happens inside the kernel.*

This project addresses that gap by:

- Showing how pod IP addresses are actually assigned at the Linux level.
- Capturing real DNAT translation in `conntrack` entries.
- Demonstrating how a NetworkPolicy translates to kernel iptables rules.
- Proving that DNS failure produces a different symptom signature than routing failure — a critical distinction for production triage.
- Documenting Raft leader election with exact log evidence and millisecond-level timing.

1.3 Why This Lab Exists

The README states clearly: *"Many engineering teams expect platform and network engineers to understand Kubernetes traffic paths and policy controls at a deep level — not just 'deploy and hope.' I built this project to understand from scratch, layer by layer, with real packet captures and real failure documentation."*

This is not a tutorial on deploying applications; it is a **network engineering investigation** into how Kubernetes actually moves packets.

1.4 Where This Fits in Real-World Engineering

Table 1: Industry roles and the skills this project demonstrates

Role	Relevant Skills from This Project
Network Engineer	Pod/service/node CIDR segmentation, CNI routing tables, NetworkPolicy as a zero-trust firewall, packet capture
Platform Engineer	Cluster provisioning, Helm, CNI lifecycle, kube-proxy, CoreDNS, etcd health, failure recovery
SRE / DevOps	Observability (Goldpinger), distributed system failure modes, conntrack exhaustion, DNS triage, Raft recovery
Security Engineer	NetworkPolicy enforcement, zero-trust segmentation, east-west traffic control, iptables audit

1.5 Why This Project Is Beyond CCNA

Your CCNA certification proves mastery of Layer 2/3 campus networking: switching, routing, ACLs, NAT, and basic troubleshooting. This project operates in a different but *related* conceptual space:

Table 2: CCNA knowledge versus what this project adds

Concept	CCNA	This Project
Subnets	Fixed subnets on physical/virtual interfaces	Dynamic per-node /26 blocks allocated by Calico IPAM
Routing	Static routes, OSPF, EIGRP on routers	Host routes per pod, programmed by CNI in Linux kernel
Firewall / ACL	Interface-level ACLs, named ACLs	Label-based NetworkPolicy enforced by Calico per-pod
NAT	Static/dynamic NAT on router	kube-proxy DNAT in iptables on every node
DNS	External DNS servers, static entries	In-cluster CoreDNS auto-wired into every pod
HA / Redundancy	STP, EtherChannel, HSRP	Raft consensus, quorum, leader election
Observability	Ping, traceroute, show commands	tcpdump in kernel namespaces, conntrack, Goldpinger

The fundamental *networking intuition* you built with CCNA (subnets, routes, firewalls, DNS, NAT) is **directly applicable** here. The difference is the layer of abstraction, the dynamism of the environment, and the distributed-systems dimension that Kubernetes adds.

Key Insight

Kubernetes does not replace networking — it **automates** networking. Every Kubernetes networking concept has an analogue in the networking you already know. The guide will always draw that analogy explicitly before introducing new terminology.

2 Prerequisite Bridge: From CCNA to This Project

2.1 What CCNA Prepares You For

The CCNA curriculum teaches you to think about networks as *static, physical, or virtualised infrastructure*. You learn to:

- Design and assign IP subnets (`/24`, `/26`, VLSM).
- Configure routing protocols (OSPF) and understand how routers exchange link-state databases to build forwarding tables.
- Apply ACLs on router or switch interfaces to permit or deny traffic.
- Configure NAT/PAT to translate private IPs to a public address.
- Set up DHCP to assign IPs dynamically.

- Understand that a DNS server resolves names to IP addresses.
- Troubleshoot connectivity using ping, traceroute, and show commands.

All of this remains relevant. The *difference* is that in Kubernetes, every one of these operations happens **automatically, dynamically, at scale, and inside the Linux kernel** of each cluster node.

2.2 The Core Conceptual Shift

In campus networking, when you want to route traffic between two VLANs, you configure an SVI on a Layer 3 switch or a sub-interface on a router. That configuration is **static** — it stays until you change it.

In Kubernetes, when a pod starts on a node, the CNI plugin:

1. Allocates an IP address from a per-node subnet block.
2. Creates a virtual ethernet pair (`veth`).
3. Moves one end of the veth into the pod's isolated network namespace.
4. Programs a host route (`ip route add $POD_IP dev cali*`) in the node's Linux routing table.
5. Removes all of this when the pod terminates.

That entire sequence happens in milliseconds, for every pod, on every node, without any human intervention. This is what "dynamic" means in the cloud-native world.

2.3 How Container Networking Differs from Campus Networking

Table 3: Campus networking vs. container networking — conceptual comparison

Aspect	Campus / Enterprise	Container / Kubernetes
Addressing unit	Host or VM, relatively static	Pod — ephemeral, new IP on every restart
Subnet assignment	Manual or DHCP pool	Per-node IPAM blocks (/26) allocated by CNI
Routing	Protocols: OSPF, EIGRP, BGP	Host routes programmed by CNI into Linux `ip route`
Firewalling	Interface ACL, VLAN-level	Per-pod iptables rules, keyed to pod labels
Load balancing	Dedicated hardware LB or F5	iptables DNAT probabilistic rules on every node
DNS	External DNS server, manual entries	CoreDNS in-cluster, auto-registered per Service
HA	STP, HSRP, VRRP	Raft consensus, leader election, quorum
Observability	SNMP, syslog, show commands	`tcpdump` in namespaces, `conntrack`, metrics
Scope	Datacenter / WAN	All on a single host (in this lab)

2.3.1 Why Pods Need Their Own IP Addresses

In a traditional network, a server has one IP address and many services run on that server differentiated by port number. This works because servers are long-lived.

A Kubernetes pod is **disposable**. It may be killed and rescheduled on a different node at any time. If services communicated with pods by IP, every consumer would need to update its connection target whenever a pod moved.

Kubernetes solves this with **Services** — virtual stable IP addresses that always route to the current set of healthy pods. The Service IP is itself not a real address; it exists only in iptables rules that rewrite packets in-flight. This is conceptually identical to DNAT in a router, just automated and applied per-pod rather than per-interface.

2.4 Why Linux Networking Matters Here

All Kubernetes networking — routing, firewalling, NAT, packet capture — is implemented **inside the Linux kernel** of each cluster node. There is no separate network device. The node's Linux kernel is the switch, the router, and the firewall simultaneously.

CCNA Bridge

Analogy: Think of each Kubernetes node as a router running IOS. The CNI plugin is like a script that runs `ip route add` and `ip link add` on that router every time a new pod starts. kube-proxy is like a NAT table that updates itself automatically. NetworkPolicy is like an extended ACL applied per-interface, except the "interface" is a virtual ethernet pair created for each pod.

The Linux networking primitives you will encounter in this project are:

`ip link`	List network interfaces (physical and virtual)
`ip addr`	List IP addresses on interfaces
`ip route`	View the kernel routing table
`iptables`	Firewall, NAT, and packet mangling rules
`conntrack`	Connection tracking table (NAT state)
`nsenter`	Enter a process's network namespace
`tcpdump`	Live packet capture

Every one of these is a direct Linux equivalent of a Cisco IOS command you already know.

2.5 Why Distributed Systems Concepts Matter

The final phase of this project deploys **etcd** — the key-value store that Kubernetes uses to persist all cluster state. etcd uses the **Raft** consensus algorithm to ensure that a write committed by the leader is replicated to a majority of nodes before being acknowledged.

This is fundamentally a distributed-systems problem, not a networking problem — but it has direct networking implications:

- If the network between etcd peers is partitioned, Raft may prevent writes until a new leader is elected.
- If too many etcd nodes fail, the cluster loses quorum and stops accepting writes — even though some nodes are still running.
- DNS in Kubernetes correctly excludes unhealthy pods from resolution, which means a failed etcd pod that is still "running" but not "ready" will be unreachable by hostname.

These failure modes require you to think simultaneously about networking ("is the packet getting there?") and distributed systems ("does the cluster have quorum?"). CCNA gives you the networking thinking; this project adds the distributed systems thinking on top.

3 Concept-by-Concept Teaching Path

This section teaches each technology used in the project in the order you need to understand it. Each concept has: a plain-English meaning, the formal technical definition, its role in this project, how to verify it, common mistakes, and references.

3.1 WSL2 — Windows Subsystem for Linux 2

3.1.1 Plain-English Meaning

WSL2 lets you run a real Linux kernel inside Windows without a full VM. You open a terminal and get Ubuntu 22.04 running on a Hyper-V lightweight VM.

3.1.2 Technical Meaning

WSL2 runs a Hyper-V micro-VM with a genuine Linux kernel (5.15 for Ubuntu 22.04). This VM has its own virtual NIC, its own IP, and its own Docker bridge network when Docker is installed.

3.1.3 The WSL2 kubectl Problem

When kind creates a cluster, each Kubernetes node is a Docker container on a Docker bridge network (`172.18.0.0/16`). The kube-apiserver inside the control-plane container listens on port 6443. Direct `kubectl` from WSL2 must cross from the WSL2 VM's user-space into Docker bridge space via port-forwarding — a path that is **unreliable in WSL2** and produces TLS timeout errors.

The fix:

```
alias k="docker exec k8s-net-lab-control-plane kubectl"
```

This runs `kubectl` inside the control-plane container where network access is direct. All phases use the `k` alias exclusively.

Warning

If you open a new terminal and forget to `source ~/.bashrc`, `k` will be undefined. All commands will fail silently. Always source before starting work.

3.1.4 Verification

```
wsl --version            # PowerShell: shows WSL version
uname -r                 # WSL: 5.15.x-microsoft-standard-WSL2
k get nodes              # Confirm alias works
```

3.1.5 Common Mistakes

Using bare `kubectl` instead of the alias; running ContainerLab simultaneously (creates conflicting Docker bridge routes — Issue 3).

3.2 Docker — Relevant Concepts for This Lab

3.2.1 Bridge Networks

Docker creates a virtual bridge interface on the host. Containers connect via veth pairs: one end in the container, the other plugged into the bridge. All containers on the same bridge can communicate at Layer 2.

CCNA Bridge

A Docker bridge network is a software VLAN-backed Layer 2 switch. Each container is a host plugged into a switch port. The bridge provides intra-VLAN communication; the host kernel provides inter-VLAN routing out.

3.2.2 Key Docker Commands in This Project

```
docker exec k8s-net-lab-control-plane kubectl ...  # k alias
docker cp manifest.yaml k8s-net-lab-control-plane:/  # Copy files
    in
docker inspect k8s-net-lab-worker --format '{{.State.Pid}}'  # Get
     PID
docker ps                                            # List
    containers
```

3.3 kind — Kubernetes IN Docker

3.3.1 What kind Creates

kind pulls a `kindest/node` Docker image (a container pre-loaded with kubelet, containerd, kube-proxy, and kubeadm). It runs one container per node role. Inside the control-plane container, it bootstraps the full Kubernetes control plane using kubeadm. All node containers share the same Docker bridge network, giving them direct IP connectivity.

3.3.2 The Cluster Config

Listing 1: kind-cluster.yaml — annotated

```yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: k8s-net-lab
networking:
  disableDefaultCNI: true          # Remove kindnet; install
     Calico manually
  podSubnet: "10.244.0.0/16"       # Pod IP space
  serviceSubnet: "10.96.0.0/12"    # Service ClusterIP space
nodes:
  - role: control-plane
    image: kindest/node:v1.29.2@sha256:51a1434a...  # SHA pin
       for WSL2 compat
  - role: worker
    image: kindest/node:v1.29.2@sha256:51a1434a...
```

```
  - role: worker
    image: kindest/node:v1.29.2@sha256:51a1434a...
```

`disableDefaultCNI: true` is the most important setting — it leaves nodes in `NotReady` state so we can observe the exact moment Calico brings them to `Ready`.

3.4 kubectl — Kubernetes API Client

`kubectl` sends HTTPS requests to the kube-apiserver, authenticated by a TLS certificate stored in `~/.kube/config`. All state changes go through this API. Key commands:

```
k get nodes                    # Node status
k get pods -A                  # All pods across all namespaces
k get pods -o wide             # Include IP and node columns
k describe pod <name>          # Detailed state + events
k logs <pod> --tail=20         # Recent container output
k exec <pod> -- <cmd>          # Execute inside a pod
k apply -f manifest.yaml       # Create or update resources
k get networkpolicies          # List NetworkPolicy objects
k get endpoints <svc>          # Pods backing a service
```

3.5 Helm — Kubernetes Package Manager

Helm renders templated Kubernetes YAML manifests from a **chart** (directory of templates + `values.yaml`) and submits them to the Kubernetes API. It is installed in Phase 1 but the primary Calico installation uses the Tigera operator pattern (direct manifests). Helm is available for more complex deployments.

3.6 Kubernetes Architecture

3.6.1 Control Plane vs Data Plane

Table 4: Kubernetes control plane vs. data plane

Component	Plane	Responsibility
kube-apiserver	Control	REST API gateway, authentication, validation
etcd	Control	Persistent store for all cluster state
scheduler	Control	Assigns pods to nodes
controller-manager	Control	Reconciliation loops (Deployments, etc.)
kubelet	Data	Starts/stops containers, calls CNI
kube-proxy	Data	Programs iptables DNAT rules for Services
Calico (CNI)	Data	Pod IP assignment, routing, NetworkPolicy

3.6.2 Architecture Diagram

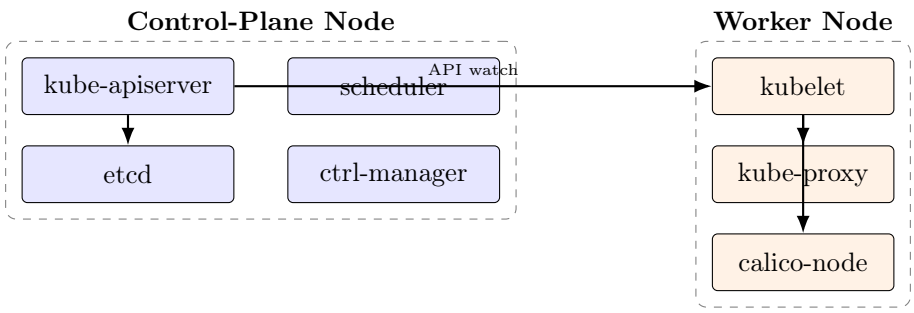

Figure 1: Kubernetes component relationships in this project

3.7 Workload Objects: Pod, Deployment, StatefulSet, DaemonSet

Pod Smallest unit. One or more containers sharing a network namespace and lifecycle. Has exactly one IP address.

Deployment Manages N identical pod replicas. If a pod dies, the controller creates a replacement. Used for stateless apps (`web-backend` in Phase 4).

StatefulSet Like a Deployment but gives pods stable identities (`etcd-0`, `etcd-1`, `etcd-2`) and ordered startup. Used for etcd in Phase 7.

DaemonSet Exactly one pod per node. Used for Calico node agents and Goldpinger.

CCNA Bridge

A Deployment is like a redundant server farm behind a load balancer. A StatefulSet is like a redundant stack of switches with persistent configuration — each unit has a fixed identity.

3.8 Services, ClusterIP, and Virtual IPs

3.8.1 The Problem

Pod IPs are ephemeral. If a pod restarts, it gets a new IP. Services provide a stable virtual IP (ClusterIP) and a DNS name that always routes to healthy pods.

3.8.2 What a ClusterIP Is

A ClusterIP does **not** exist on any interface or host. It exists only in iptables DNAT rules on every node. Any packet destined to the ClusterIP is intercepted and rewritten to a real pod IP before leaving the node. The pod receiving the request replies to the original source; the kernel uses the conntrack table to restore the ClusterIP as the source on the return path.

CCNA Bridge

ClusterIP is a virtual IP in a NAT policy: "translate dst 10.96.x.y:80 to one of {podA, podB, podC} with equal-cost load balancing." The difference is that kube-proxy programs this automatically on every node whenever pod readiness changes.

3.9 kube-proxy and iptables DNAT

kube-proxy watches the API server for Service and Endpoint changes. For each Service it creates iptables chains:

- `KUBE-SVC-XXXX` — entry chain for a specific ClusterIP:port
- `KUBE-SEP-YYYY` — per-pod endpoint with DNAT rule

Probabilistic load balancing: with 3 pods:

- Rule 1: match probability 0.333 → DNAT to pod 1
- Rule 2: match probability 0.5 of remainder → DNAT to pod 2
- Rule 3: remainder → DNAT to pod 3

Net result: 33.3% each. When pods appear or disappear, kube-proxy atomically replaces all rules.

3.10 CoreDNS and Kubernetes DNS

CoreDNS runs as a Deployment in `kube-system`, exposed via a Service with ClusterIP `10.96.0.10`. kubelet injects into every pod's `/etc/resolv.conf`:

```
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

FQDN format: `<service>.<namespace>.svc.cluster.local`

The `ndots:5` setting means every name with fewer than 5 dots is first tried with search domains appended. Combined with the `A` and `AAAA` dual lookup behaviour, every `curl web-service` call generates **at minimum 6 DNS queries** before resolving. This is visible in the Phase 5 captures.

Headless services and StatefulSets: The etcd Service in Phase 7 is a **headless service** (`clusterIP: None`). Instead of a single VIP, CoreDNS returns individual pod IPs for each member. This is how `etcd-0.etcd.default.svc.cluster.local` resolves directly to the `etcd-0` pod IP.

3.11 CNI — Container Network Interface Specification

CNI is a specification (not a product) that defines how kubelet should invoke a network plugin binary when a pod starts (`cmdAdd`) or stops (`cmdDel`). The plugin receives pod metadata as JSON and must return the assigned IP address. Without a CNI, nodes remain `NotReady` because kubelet cannot assign IPs or verify pod health.

3.12 Calico CNI

3.12.1 IPAM: Per-Node /26 Blocks

- Total space: `10.244.0.0/16` (65,536 IPs).
- Each node: `/26` block (64 IPs).
- Evidence: pods on different nodes have IPs from different /26 ranges.

3.12.2 veth Pairs

For each pod, Calico creates a veth pair: `eth0` inside the pod namespace, `cali*` on the host. The host has a route: `ip route add <pod-ip> dev cali*`.

3.12.3 Cross-Node Routing (VXLANCrossSubnet)

```
On control-plane node     routes programmed by Calico (proto 80):
blackhole 10.244.139.128/26 proto 80      # Own block, no host
   route = blackhole
10.244.139.129 dev cali512f322e769        # Direct route to local
    pod
10.244.11.0/26 via 172.18.0.7 dev eth0   # Worker2 pod range via
   Docker bridge
10.244.134.64/26 via 172.18.0.6 dev eth0 # Worker pod range
```

The `via 172.18.0.x` addresses are the Docker bridge IPs of the other kind node containers. Traffic to another node's pod subnet transits through the Docker bridge — no real VXLAN encapsulation needed since all nodes share the same L2 segment.

3.12.4 Calico and NetworkPolicy

Calico enforces NetworkPolicy by translating each policy into iptables chains on the relevant nodes. Rules are keyed to pod labels, not IP addresses — important because pod IPs change but labels do not.

3.13 Linux Network Namespaces and veth Pairs

3.13.1 Network Namespaces

A Linux network namespace is an isolated instance of the network stack: separate interfaces, routing table, iptables, and ports. Each pod runs in its own namespace.

CCNA Bridge

Network namespaces are equivalent to VRF instances on a Cisco router. Each VRF has its own routing table, and interfaces must be explicitly placed in a VRF. Network namespaces work identically.

3.13.2 veth Pairs

A veth pair is a bidirectional virtual cable. What enters one end exits the other. One end in the pod namespace (`eth0`), one end in the host namespace (`cali*`). The host's end has a /32 route pointing at it.

3.14 nsenter — Entering Namespaces from the Host

kind nodes are minimal containers without diagnostic tools. `nsenter` lets you run tools from the WSL2 host inside the node container's network namespace:

```
NODE_PID=$(docker inspect k8s-net-lab-worker --format '{{.State.
   Pid}}')
sudo nsenter -t $NODE_PID -n tcpdump -i eth0 -n  # -n = enter net
   namespace
sudo nsenter -t $NODE_PID -n iptables -t nat -L -n
sudo nsenter -t $NODE_PID -n conntrack -L 2>/dev/null
sudo nsenter -t $NODE_PID -n ip route
```

3.15 tcpdump — Packet Capture

`tcpdump` via `nsenter` in Phase 5 reveals:

- DNAT in action: the same packet appears with ClusterIP destination, then again with the real pod IP destination — the translation is visible at the interface level.
- Full TCP lifecycle: `[S]` SYN, `[S.]` SYN-ACK, `[.]` ACK, `[P.]` data, `[F.]` FIN.

- DNS dual A+AAAA queries: every `nslookup web-service` fires two queries simultaneously.

Useful flags: `-i any` (all interfaces), `-n` (no name resolution), `-c N` (stop after N packets), `port 80`, `port 53`.

3.16 conntrack — Connection State Tracking

The Linux netfilter subsystem tracks every NATed connection. Each entry stores:

- **Original tuple:** pod → ClusterIP
- **Translated tuple:** pod → real pod IP

When the server replies, the kernel finds the entry and rewrites the source IP back to the ClusterIP before the client sees it.

Warning

When `conntrack -C` (the table count) reaches `nf_conntrack_max` (≈262,144 by default), all new connections are **silently dropped**. No error, no RST — just connection timeouts. This is a common cause of mysterious production outages under high load.

3.17 NetworkPolicy and Zero-Trust Segmentation

3.17.1 How NetworkPolicy Works

- Policies are **additive** (allow-only). There is no explicit deny rule — you build a default-deny foundation and layer allows on top.
- A pod with *no matching policy* gets all traffic allowed.
- A pod with *any matching policy* gets only what that policy allows.
- Blocked traffic is **silently dropped**.

3.17.2 3-Tier Architecture in Phase 3

Listing 2: Default-deny foundation in Phase 3

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
spec:
  podSelector: {}        # Matches ALL pods in namespace
  policyTypes:
  - Ingress
  - Egress
  # No rules = nothing allowed
```

Traffic matrix after all policies are applied:

Table 5: Allowed traffic paths in the 3-tier NetworkPolicy setup

Source → Dest	**frontend**	**backend**	**database**
frontend	—	✓	×
backend	×	—	✓
database	×	×	—

3.18 Goldpinger

Bloomberg's cross-node health checker. Deployed as a DaemonSet. Each pod periodically HTTP-pings all other Goldpinger pods. The `/check_all` endpoint returns a JSON matrix of reachability between all node pairs. A response of `"OK": true` with `"response-time-ms": 1` confirms sub-millisecond cross-node pod networking.

Requires RBAC: a ServiceAccount with a ClusterRole that allows `list/watch/get` on `pods` — needed to discover peer IPs via the Kubernetes API.

3.19 etcd

A distributed, strongly-consistent key-value store. In production Kubernetes, etcd holds all cluster state. In Phase 7, a standalone 3-node etcd cluster is deployed inside the kind cluster as pods to study Raft behaviour. Ports: `2379` (client), `2380` (peer).

3.20 Raft Consensus Algorithm

3.20.1 Key Terms

Leader Accepts all writes. Replicates log entries to followers.

Follower Accepts log entries; responds to heartbeats.

Candidate A follower campaigning for leadership.

Term Monotonically increasing election epoch. Increment = election.

Quorum Majority of nodes. 3-node cluster: quorum = 2.

Commit A write is committed once a quorum acknowledges it.

3.20.2 Election Timeline (from Phase 7 Logs)

```
T+0ms  Leader deleted / isolated
T+0ms  received MsgTimeoutNow from etcd-0 (graceful stepdown)
T+4ms  etcd-1: starting new election at term 3
T+4ms  etcd-1: became candidate
T+4ms  etcd-1: sent MsgVote to etcd-2
T+4ms  etcd-1: received MsgVoteResp (2 votes = quorum)
T+4ms  etcd-1: became leader at term 3
```

3.20.3 Quorum Table

Table 6: Raft quorum and fault tolerance

Nodes	Quorum	Tolerable Failures	Note
1	1	0	No HA
2	2	0	No HA
3	2	1	This lab
5	3	2	Production min

3.21 Comparison Tables

Table 7: Docker vs. kind vs. Kubernetes — what each layer provides

Aspect	Docker	kind	Kubernetes
Abstraction	Containers on one host	Multi-node K8s in Docker	Distributed container platform
Networking	Docker bridge / veth	Docker bridge between nodes	Pod/Service/CNI abstraction
Scheduling	None	None	Scheduler + kubelet
Health	Restart policies	Uses K8s health	Readiness/liveness probes
Storage	Volumes	Ephemeral (no PVC)	PersistentVolumeClaims
DNS	Container names	CoreDNS	CoreDNS, FQDN per service

Table 8: Network address spaces in this lab

Network	CIDR	Assigned by	Purpose
Node (Docker bridge)	172.18.0.0/16	Docker	Node-to-node container IPs
Pod	10.244.0.0/16	Calico IPAM	One IP per pod
Service (ClusterIP)	10.96.0.0/12	kube-apiserver	Virtual IPs for Services
Host (WSL2)	varies	Windows/Hyper-V	WSL2 VM NIC

Table 9: Calico vs. kube-proxy vs. CoreDNS — distinct responsibilities

Component	Responsibility	Mechanism
Calico	Pod IP assignment, cross-node routing, NetworkPolicy	ip route, veth, iptables/eBPF
kube-proxy	ClusterIP → pod IP translation (load balancing)	iptables DNAT chains
CoreDNS	Service name → ClusterIP resolution	DNS server, K8s API watch

Table 10: iptables (Calico) vs. eBPF (Cilium) — architectural difference

Property	iptables / Calico	eBPF / Cilium
Rule evaluation	Sequential chain traversal	Direct bytecode, O(1)
Performance at scale	Degrades with rule count	Constant time
Observability	Limited (iptables -L)	Full flow visibility
kube-proxy needed	Yes	No (replaced entirely)
WSL2 stability	Stable	Limited (kernel requirements)
Production adoption	Widespread	Growing rapidly

4 Repository Walkthrough

This section walks through every file in the repository, explaining its purpose, important commands, interactions with other phases, and any fragility or workaround content.

4.1 Repository Structure

```
k8s-networking-project/
          README.md                    # Project overview, goals,
    tool list
          phase1-environment-setup.md  # Install tools, create
   cluster
          phase2-calico-cni.md         # Install Calico, verify
   pod networking
          phase3-networkpolicy.md      # Zero-trust segmentation
          phase4-services-dns.md       # ClusterIP, kube-proxy,
   CoreDNS
          phase5-packet-capture.md     # tcpdump, conntrack via
   nsenter
          phase6-observability.md      # Goldpinger DaemonSet
          phase7-raft-failures.md      # etcd Raft cluster + 3
   failure modes
```

```
          issues.md                    # 10 documented issues
   with root cause + fix
```

4.2 README.md

Type: Overview and navigation document.

The README establishes the goal (*"Convert Kubernetes comfort into network engineer value"*), lists the tech stack, summarises each phase in a table, provides quick-start instructions, calls out the WSL2 kubectl alias requirement, and lists all tools with their purpose.

Notable design decision: The README does not attempt to be a comprehensive tutorial. It is a navigational document pointing to phase files. This is a professional pattern: separate concern (overview vs. detail).

The WSL2 warning is prominently placed:

```
WSL2 Note: kubectl direct access is broken on WSL2.
Use: alias k="docker exec k8s-net-lab-control-plane kubectl"
```

This warning prevents the most common failure mode before the reader even starts.

Failure modes table: The README summarises the three Raft failure modes — leader loss, NotReady DNS exclusion, and bootstrap conflict — as first-class results of the project. This elevates distributed-systems knowledge as an explicit deliverable, not an afterthought.

4.3 phase1-environment-setup.md

Type: Setup / infrastructure.

Purpose: Install Docker, kubectl, kind, Helm, and networking tools on a fresh Ubuntu 22.04 WSL2 system. Create the 3-node kind cluster.

4.3.1 Key Steps and Their Purpose

Step 1 — System update and dependencies: Standard `apt` installs. Unremarkable but necessary.

Step 2 — Docker installation: Full Docker CE install from the official repository. The user is added to the `docker` group so `sudo` is not needed for `docker` commands.

Step 3 — kubectl installation: Installs from the Kubernetes apt repo, pinned to v1.29. Important that kubectl version matches the kind node image version.

Step 4 — kind installation: Downloaded as a binary from the kind GitHub releases. Version pinned to v0.22.0.

Step 5 — Helm installation: Downloaded via the official Helm install script.

Step 6 — Networking tools:

```
sudo apt install -y tcpdump conntrack iputils-ping iproute2
   dnsutils net-tools
```

This installs the diagnostic tools that will be used via `nsenter` from the WSL2 host in Phase 5.

Step 7 — Cluster creation: Creates `kind-cluster.yaml` with the settings detailed in Section 3.3 and runs `kind create cluster`. Nodes will be `NotReady` at this point — intentional.

Step 8 — WSL2 kubectl fix: Adds the `k` alias to `.bashrc`.

4.3.2 Fragility Notes

- The SHA hash pin for the node image is environment-specific but correct. Other SHA hashes can be found at the kind release pages.
- The `kind version` pinned to v0.22.0 in Phase 1 differs from the README's "v0.31.0" reference. Minor inconsistency.
- The expected output section shows `NotReady` with a note that this is intentional — this is excellent pedagogical practice.

4.4 phase2-calico-cni.md

Type: Networking — CNI installation and verification.

Purpose: Install Calico as the cluster CNI, observe nodes transition from `NotReady` to `Ready`, and verify cross-node pod communication with packet capture evidence.

4.4.1 Step-by-Step Analysis

Step 1 — Tigera operator:

```
k create -f https://raw.githubusercontent.com/projectcalico/calico
   /v3.27.0/manifests/tigera-operator.yaml
```

The Tigera operator is a Kubernetes controller that manages the Calico installation lifecycle. It watches for an `Installation` CR (custom resource) and deploys Calico accordingly.

Step 2 — Installation CR:

```
spec:
  calicoNetwork:
    ipPools:
    - blockSize: 26           # /26 per node = 64 pod IPs per
       node
      cidr: 10.244.0.0/16     # Must match kind-cluster.yaml
         podSubnet
      encapsulation: VXLANCrossSubnet  # VXLAN only when
         crossing subnet boundaries
      natOutgoing: Enabled    # SNAT pod traffic leaving the
         cluster
      nodeSelector: all()     # Apply to all nodes
```

Critical match: `cidr: 10.244.0.0/16` must exactly match `podSubnet` in `kind-cluster.yaml`. If they differ, Calico will allocate IPs outside the range kube-proxy expects, breaking service routing.

Step 3 — Watching the transition: The document instructs the reader to open a second terminal and watch `k get nodes -w` while Calico installs. This teaches the fundamental relationship: *CNI healthy ⇒ nodes become Ready.*

Steps 4–7 — Verification:

- Inspecting `ip route` on the control-plane to see Calico-programmed routes (`proto 80`) to each worker's pod subnet.
- Viewing `cali*` veth interfaces to confirm pod-to-node wiring.
- Deploying `pod-a` (on worker) and `pod-b` (on worker2) and pinging across nodes.
- `ttl=62` on the ping reply proves cross-node routing (2 hops decremented from TTL 64).
- Live `tcpdump` on the worker during the ping showing pod IPs (no NAT between pods).

4.4.2 What This Phase Proves

The combination of routing table inspection + ping + packet capture constitutes **empirical proof** that the CNI is working correctly. This is the standard methodology for network validation.

4.5 phase3-networkpolicy.md

Type: Networking — security / segmentation.

Purpose: Build and verify a zero-trust 3-tier application network.

4.5.1 Step-by-Step Analysis

Step 1 — Baseline documentation: The phase begins by proving the default-open state before applying any policy. This establishes a baseline — essential for before/after comparison.

Step 2 — Default-deny-all:

```
spec:
  podSelector: {}        # Empty selector = all pods in
     namespace
  policyTypes:
  - Ingress
  - Egress
  # No ingress/egress rules = nothing allowed
```

This immediately blocks all traffic. The verification command confirms `ping` times out.

Step 3 — 3-tier deployment: Three pods with labels `tier: frontend`, `tier: backend`, `tier: database`. The label is the key — NetworkPolicy matches on labels, not on pod names or IPs.

Step 4 — Allow policies: Four separate NetworkPolicy objects:

- `allow-frontend-to-backend`: guards backend ingress
- `allow-backend-to-database`: guards database ingress
- `allow-backend-egress`: allows backend to reach database
- `allow-frontend-egress`: allows frontend to reach backend

Step 5 — Full matrix verification: Tests all four paths: two allowed (pass), two blocked (fail). The expected output is documented explicitly, including the correct error messages.

Step 6 — Policy audit: `kubectl describe networkpolicy` to confirm what is enforced, plus a `nsenter` + `iptables -L` to see the raw Calico iptables chains.

4.5.2 Fragility Notes

- The phase creates pods without a Deployment — they must be manually recreated if the cluster is restarted.
- Step 6 includes viewing iptables rules via nsenter but does not explain the specific chain names (`cali-FORWARD`, `cali-pi-...`) — a gap for the reader.
- The label-mismatch issue (Issue 7) is referenced, which is good.

4.6 phase4-services-dns.md

Type: Networking — services, load balancing, DNS.

Purpose: Deploy a ClusterIP Service, inspect kube-proxy DNAT rules, trace CoreDNS, and reproduce the DNS failure signature.

4.6.1 Step-by-Step Analysis

Step 1 — Web service deployment: A 3-replica nginx Deployment plus a ClusterIP Service. This creates the conditions for load balancing.

Step 2 – Step 3 — Proving ClusterIP is virtual:

```
CLUSTER_IP=$(k get svc web-service -o jsonpath='{.spec.clusterIP
   }')
docker exec k8s-net-lab-control-plane kubectl exec client -- curl
   http://$CLUSTER_IP
```

Traffic works even though no machine owns the ClusterIP — proof of iptables DNAT.

Step 4 — iptables DNAT inspection:

```
sudo nsenter -t $WORKER_PID -n iptables -t nat -L KUBE-SERVICES -n
    | grep web
sudo nsenter -t $WORKER_PID -n iptables -t nat -L -n | grep -A 4 "
   $CLUSTER_IP"
```

Expected output shows the probabilistic 0.333 / 0.5 / 1.0 DNAT rules for three pods. This is the core empirical evidence for how kube-proxy works.

Step 5 — DNS trace: `dig web-service.default.svc.cluster.local` shows the A record returning the ClusterIP, answered by `10.96.0.10` (CoreDNS). The `/etc/resolv.conf` dump shows the auto-injected search domains.

Step 6 — DNS failure simulation: A NetworkPolicy blocks UDP/53 egress from the client pod. Result:

- `nslookup web-service` times out.
- `curl web-service` fails.

- `curl $CLUSTER_IP` succeeds.

This is the **DNS failure signature** — one of the most valuable diagnostic patterns in the entire project. It proves the service infrastructure is healthy and isolates DNS as the single point of failure.

DNS Failure Triage Decision Tree: Phase 4 includes an ASCII decision tree that is reproduced in Section 6 of this guide.

4.7 phase5-packet-capture.md

Type: Observability — packet-level analysis.

Purpose: Perform live packet capture using `tcpdump` and inspect the `conntrack` table via `nsenter`, without installing any tools on the nodes.

4.7.1 Step-by-Step Analysis

Step 1 — Continuous traffic loop: Terminal 2 runs a 1-second `curl` loop. This keeps traffic flowing during captures — a technique that mimics real production traffic scenarios.

Step 2 – Step 3 — HTTP capture:

```
sudo nsenter -t $CLIENT_NODE_PID -n tcpdump -i any port 80 -n -c
   20
```

The output explanation is detailed and teaches DNAT translation visibility: the packet appears on `cali*` with ClusterIP as destination, then on `eth0` with the real pod IP after DNAT.

Step 4 — DNS capture: Capturing port 53 traffic while running `nslookup web-service` shows: the DNS query going to the CoreDNS ClusterIP, the DNAT to the real CoreDNS pod IP, and the A record response containing the web-service ClusterIP.

Step 5 — conntrack:

```
sudo nsenter -t $NODE_PID -n conntrack -L 2>/dev/null | grep "
   $CLUSTER_IP"
sudo nsenter -t $NODE_PID -n conntrack -C 2>/dev/null  # Total
   count
```

The output explanation covers reading the original and translated tuples. The warning about `nf_conntrack_max` exhaustion is correctly included.

Step 6 — End-to-end trace: Captures both port 80 and port 53 simultaneously for one HTTP request, showing the complete sequence: DNS query → DNS answer → TCP connect → HTTP GET → HTTP 200 → close.

4.7.2 Quick Reference: nsenter Patterns

Phase 5 includes a comprehensive nsenter quick-reference section covering all the `tcpdump`, `conntrack`, and `iptables` patterns used throughout the project. This section is reused in Section 8 of this guide.

4.8 phase6-observability.md

Type: Observability — cross-node connectivity monitoring.

Purpose: Deploy Goldpinger as a DaemonSet for continuous cross-node health monitoring. Introduces eBPF as a conceptual contrast to iptables.

4.8.1 eBPF vs iptables Section

Phase 6 begins with a conceptual comparison of iptables (what Calico uses in this lab) and eBPF (what Cilium uses in production). This is explicitly framed as *architectural vocabulary* rather than a practical exercise, since Cilium is unstable on WSL2. The comparison table (Section 3.21) captures this material.

4.8.2 Goldpinger Deployment Analysis

RBAC (Step 1):

```
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["list", "watch", "get"]
```

Goldpinger calls the Kubernetes API to discover the IPs of peer pods. Without this RBAC, the `/check_all` endpoint returns `"hosts": null`.

DaemonSet (Step 2): The `POD_IP` environment variable is injected from the pod's own IP using the Kubernetes Downward API:

```
- name: POD_IP
  valueFrom:
    fieldRef:
      fieldPath: status.podIP
```

Step 5 — Network partition simulation: A NetworkPolicy isolates one Goldpinger pod, and the `/check_all` response immediately shows `"OK": false` for that pod. Removing the policy restores `"OK": true`. This is the most impressive single demonstration in the project: real-time observability reflecting a network policy change.

4.9 phase7-raft-failures.md

Type: Distributed systems — consensus, failure modes.

Purpose: Deploy a standalone 3-node etcd cluster inside the Kubernetes lab, then deliberately cause three failure modes with documented evidence and recovery procedures.

4.9.1 etcd StatefulSet Analysis

Headless service: `clusterIP: None`. This causes CoreDNS to return individual pod IPs rather than a single VIP. This is how `etcd-0.etcd` resolves to the specific `etcd-0` pod IP.

Pod naming in etcd commands:

```
- name: POD_NAME
  valueFrom:
    fieldRef:
      fieldPath: metadata.name  # "etcd-0", "etcd-1", "etcd-2"
```

Used to construct the `-name`, `-initial-advertise-peer-urls`, and `-advertise-client-urls` flags dynamically.

No PVCs (intentional): The StatefulSet deliberately omits `volumeClaimTemplates`. This is the root cause of Failure Mode 3 and is documented as such.

4.9.2 Failure Mode 1 — Leader Node Loss

Two options: NetworkPolicy isolation (softer) or pod deletion (harder). Both trigger Raft election. The documentation includes:

- Exact log evidence showing the election sequence
- RAFT TERM increment as observable evidence
- Zero data loss verification by reading back the pre-crash key
- Sub-4ms election time measurement

4.9.3 Failure Mode 2 — DNS Exclusion of NotReady Pods

The most subtle failure: a pod that is `Running` but `Ready=0/1` is *correctly* excluded from DNS. This is not a bug. The documentation explains the Kubernetes semantics: headless service DNS only includes `ReadyAddresses`, not `NotReadyAddresses`.

4.9.4 Failure Mode 3 — Bootstrap Conflict

The etcd `-initial-cluster-state=new` flag conflicts with an existing data directory. The cascade: crash loop → NotReady → DNS exclusion → cluster degraded on 2/3 nodes. Recovery requires full StatefulSet deletion and redeploy.

4.10 issues.md

Type: Troubleshooting / reference.

Purpose: Document 10 issues encountered during the project with exact symptoms, root cause analysis, and working fixes.

Table 11: issues.md — all 10 issues summarised

#	Phase	Summary
1	Phase 1	kind fails: cgroup error — use SHA-pinned v1.29.2 image
2	All	kubectl TLS timeout on WSL2 — use `docker exec` alias
3	Phase 1, 6	ContainerLab Docker networks conflict with kind bridge
4	Phase 2+	EOF heredoc piping broken with `k` alias — use `docker cp`
5	Phase 2, 6	CoreDNS crash-loops if CNI not fully ready — restart coreDNS
6	Phase 5	tcpdump not found on kind node — use nsenter from host
7	Phase 3, 4	NetworkPolicy does nothing — label mismatch on podSelector
8	Phase 7	etcd bootstrap conflict — no PVCs, data dir persists
9	All	Stale kubeconfig after cluster recreate — re-export or use alias
10	Phase 7	`kubectl exec` fails on crash-looping pod — use crictl or redeploy

Professional quality of issues.md: Each issue follows the same structure: phase, symptom (with exact error text), root cause, fix (with exact commands), and prevention advice. Issue 8 even includes the correct production YAML for PVCs. This document alone demonstrates serious debugging skill to an interviewer.

4.10.1 Issue 4 — EOF Piping Limitation

This is a subtle and important issue. The `k` alias runs `docker exec`, which does not inherit stdin from the shell. The fix:

```
# WRONG -- stdin not passed through docker exec
cat <<EOF | k apply -f -
...
EOF

# CORRECT -- copy file into container first
cat <<EOF > manifest.yaml
...
EOF
docker cp manifest.yaml k8s-net-lab-control-plane:/manifest.yaml
docker exec k8s-net-lab-control-plane kubectl apply -f /manifest.
   yaml
```

This pattern is used *throughout* the project. Understanding why it is necessary (the docker exec stdin limitation) explains the seemingly verbose approach.

5 End-to-End Architecture and Workflow

5.1 The Full Stack in This Lab

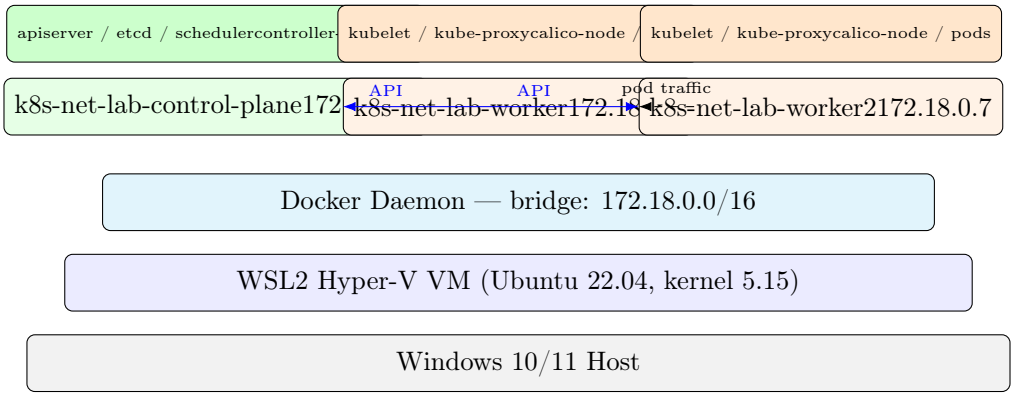

Figure 2: Complete WSL2 → Docker → kind → Kubernetes stack

5.2 Phase-by-Phase State Transitions

5.2.1 After Phase 1: Cluster with No CNI

- Three Docker containers running as "nodes."
- Control plane is up: apiserver, scheduler, etcd, controller-manager.
- All nodes: `NotReady` — no CNI ⇒ no pod networking.
- CoreDNS pods are `Pending` — they cannot be scheduled because the node is not ready.

5.2.2 After Phase 2: Calico Installed

- Tigera operator and Calico node DaemonSet running.
- Each node gets a `/26` pod subnet assigned.
- Routing table on each node has routes to other nodes' pod subnets.
- All nodes: `Ready`.
- CoreDNS starts. Cluster is fully functional.
- Test pods communicate across nodes. TTL evidence confirms routing.

5.2.3 After Phase 3: Zero-Trust Segmentation

- Default-deny-all NetworkPolicy blocks all pod-to-pod traffic.
- Explicit allow policies permit only frontend → backend → database.
- Calico has programmed iptables rules on every node to enforce this.

5.2.4 After Phase 4: Services and DNS

- A ClusterIP Service fronts 3 nginx replicas.
- kube-proxy has programmed DNAT rules on every node.
- CoreDNS has registered the service FQDN.
- DNS failure has been reproduced and diagnosed.

5.2.5 After Phase 5: Observability

- DNAT translation visible in live tcpdump.
- conntrack DNAT pairs inspected.
- Full HTTP lifecycle (DNS → TCP → HTTP) captured in one trace.

5.2.6 After Phase 6: Goldpinger

- One Goldpinger pod per node.
- Cross-node health data available via JSON API.
- Network partition demonstrated and health check reflects it.

5.2.7 After Phase 7: etcd Raft

- Standalone 3-node etcd cluster deployed as StatefulSet.
- Three failure modes reproduced and documented.
- Raft election evidence in logs.
- Zero data loss after leader failure confirmed.

5.3 Why Nodes Are NotReady Before CNI

When kubelet starts on a node, it checks the CNI health as part of the node condition assessment. Without a CNI:

1. kubelet cannot assign IP addresses to pods.
2. Pods cannot be started on the node.
3. kubelet cannot confirm that the pod network is functional.
4. The `Ready` condition is not met.

The controller-manager's node lifecycle controller marks the node `NotReady`. The scheduler will not place pods on `NotReady` nodes. CoreDNS pods (which need to be scheduled) remain `Pending`. The cluster is in a "chicken and egg" state: DNS needs a node, a node needs CNI, CNI needs to be installed.

5.4 How Pod IP Addresses Are Assigned

1. User runs `kubectl apply -f pod.yaml`.
2. kube-apiserver stores the pod object in etcd.
3. The scheduler picks a node and writes `spec.nodeName`.
4. kubelet on that node sees the assignment via API watch.
5. kubelet calls the container runtime (containerd) to start the container.
6. kubelet calls the CNI plugin binary with the pod metadata.
7. Calico allocates the next available IP from the node's /26 block.
8. Calico creates the veth pair and configures the pod's network namespace.
9. Calico programs the host route: `ip route add $POD_IP dev cali*`.
10. The pod's IP is written back to the pod object in the API.
11. `kubectl get pod -o wide` now shows the IP.

5.5 How Pod-to-Pod Communication Works

5.5.1 Same Node

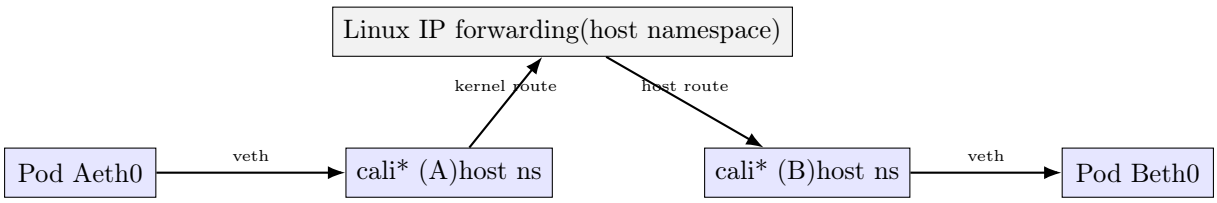

Figure 3: Same-node pod-to-pod traffic path via Linux IP forwarding

1. Pod A sends a packet to Pod B's IP.
2. The packet exits Pod A's `eth0` into the host namespace via the veth pair.
3. The host kernel looks up the routing table. Calico has a host route: `10.244.x.y dev cali*(B)`.
4. The packet is forwarded directly to Pod B's veth interface.
5. No NAT, no tunneling.

5.5.2 Cross-Node

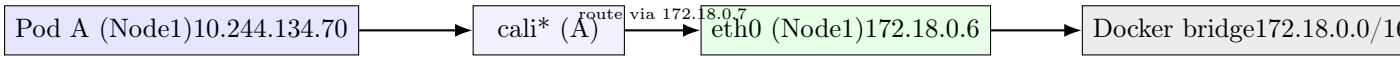

Figure 4: Cross-node pod routing via Docker bridge (Calico programs the route)

Calico has programmed: `10.244.11.0/26 via 172.18.0.7 dev eth0`. The packet from Pod A goes out the node's `eth0` toward Node2's Docker bridge IP. Node2 receives it and forwards it to Pod B via the `cali*(B)` veth. TTL is decremented by 2 — visible in the Phase 2 ping output (`ttl=62`).

5.6 How Services Work

When a pod sends traffic to a ClusterIP:

1. The packet exits the pod's `eth0`, enters the host namespace.
2. iptables `PREROUTING` chain intercepts it.
3. The `KUBE-SVC` chain probabilistically selects a pod endpoint.
4. A `KUBE-SEP` rule DNATs the destination: ClusterIP → real pod IP.
5. A conntrack entry records the translation pair.
6. The packet is routed to the real pod (same-node or cross-node).
7. The real pod replies to the original source (the client pod's IP).
8. The kernel finds the conntrack entry and rewrites the source from real pod IP back to ClusterIP.
9. The client pod receives a reply apparently from the ClusterIP.

5.7 How DNS Works

1. Pod makes a DNS query for `web-service`.
2. `/etc/resolv.conf` has `ndots:5` and search domains.
3. Name has 0 dots $< 5 \Rightarrow$ try appended: `web-service.default.svc.cluster.local`.
4. A and AAAA queries go to `10.96.0.10` (CoreDNS ClusterIP).
5. iptables DNATs `10.96.0.10:53` to a real CoreDNS pod IP.
6. CoreDNS looks up the service, finds the ClusterIP, returns the A record.
7. The ClusterIP is in the DNS response.
8. The pod connects to the ClusterIP and kube-proxy handles the rest.

5.8 How NetworkPolicy Changes Traffic

Without any policy, all pod-to-pod traffic is allowed via the Linux IP forwarding path. When Calico programs a NetworkPolicy:

1. Calico translates the policy into iptables rules on the relevant nodes.
2. Rules are inserted in `cali-FORWARD`, `cali-INPUT`, and per-pod chains.
3. Packets are matched against the pod's labels (via iptables marks set on the veth interface).
4. Non-matching packets are dropped silently (no RST).

5.9 How etcd and Raft Relate to Cluster Behaviour

Every Kubernetes operation that changes state (create pod, update deployment, apply NetworkPolicy) results in a write to etcd. If etcd loses quorum:

- The kube-apiserver cannot accept write requests.
- The scheduler cannot schedule new pods.
- Controllers cannot reconcile — failed pods are not replaced.
- **Existing pods continue running.** The data plane is unaffected.

This distinction — control plane down, data plane still running — is a critical operational concept. A cluster with a failed control plane is not "down" for existing workloads. It is only unable to *change* state.

6 End-to-End Packet and Traffic Walkthroughs

This section provides detailed, step-by-step traffic walkthroughs for the key scenarios demonstrated in this project. For each scenario: where traffic starts, which components handle it, which Linux/Kubernetes mechanisms are used, and which commands reveal it.

6.1 kubectl / API Access Path in This WSL2 + kind Setup

Normal path (broken on WSL2):

```
kubectl (WSL2 userspace)
  -> /etc/kubectl config: server: https://127.0.0.1:PORT
  -> port-forward from WSL2 VM to Docker container port 6443
  -> FAILS: WSL2 port-forwarding to Docker bridge is unreliable
  -> Result: TLS handshake timeout
```

Working path (the k alias):

```
k get nodes
  -> shell expands alias to: docker exec k8s-net-lab-control-
     plane kubectl get nodes
  -> docker exec enters the control-plane container process
     namespace
```

```
  -> kubectl runs INSIDE the container
  -> localhost:6443 = the apiserver's port inside the container
  -> direct connection, no port-forwarding
  -> Result: response in <100ms
```

Observation commands:

```
docker ps | grep control-plane       # Confirm container is
   running
docker inspect k8s-net-lab-control-plane | grep -i pid  #
   Container PID
k get nodes                          # Confirm alias works
```

6.2 Pod on Node A to Pod on Node A (Same-Node)

Scenario: Pod A (10.244.134.70) pings Pod B (10.244.134.71) — both on worker node.

1. Pod A's kernel looks up the destination IP in its routing table. Default route points to `169.254.1.1` (Calico's link-local gateway on the veth).
2. The packet exits Pod A's `eth0` into the host namespace via the `cali*(A)` veth.
3. The host kernel receives the packet on `cali*(A)` and performs an IP route lookup. Calico has: `10.244.134.71 dev cali*(B)`.
4. The packet is forwarded to `cali*(B)`, enters Pod B's namespace.
5. Pod B receives the packet on its `eth0`.
6. Pod B's reply follows the reverse path.

TTL behaviour: No TTL decrement occurs from the pod's perspective (the host-level forwarding does not decrement TTL within a namespace boundary in this configuration). `ttl=64` in the reply.

Observation commands:

```
# Check routing table inside Pod A
k exec pod-a -- ip route

# Capture on worker node - both pod IPs visible, no VXLAN
WORKER_PID=$(docker inspect k8s-net-lab-worker --format '{{.State.
   Pid}}')
sudo nsenter -t $WORKER_PID -n tcpdump -i any icmp -n -c 10
```

6.3 Pod on Node A to Pod on Node B (Cross-Node)

Scenario: Pod A (10.244.134.70, on worker) pings Pod B (10.244.11.2, on worker2).

1. Pod A sends to 10.244.11.2. Default route via veth.
2. Packet enters host namespace on `cali*(A)`.

3. Route lookup: `10.244.11.0/26 via 172.18.0.7 dev eth0`. (Calico programmed this. 172.18.0.7 is worker2's Docker bridge IP.)
4. Packet exits worker's `eth0` onto the Docker bridge.
5. Docker bridge delivers to worker2's `eth0` (172.18.0.7).
6. worker2's routing table: `10.244.11.2 dev cali*(B)`.
7. Packet delivered to Pod B via veth.

TTL behaviour: TTL decrements by 2 (one on each node). Observable as `ttl=62` in the ping reply from Pod B (started at 64).

VXLAN: In this lab, all nodes are on the same L2 Docker bridge, so `VXLANCrossSubnet` does not encapsulate. In a real multi-subnet cluster, Calico would add a VXLAN header.

Observation commands:

```
# Show Calico-programmed routes (proto 80 = Calico)
docker exec k8s-net-lab-worker ip route | grep "proto 80"

# Capture ICMP on worker's eth0 - shows pod IPs, no NAT
WORKER_PID=$(docker inspect k8s-net-lab-worker --format '{{.State.
   Pid}}')
sudo nsenter -t $WORKER_PID -n tcpdump -i eth0 icmp -n

# Expected output:
# 10.244.134.70 > 10.244.11.2: ICMP echo request   <- pod-a
# 10.244.11.2   > 10.244.134.70: ICMP echo reply   <- pod-b
```

6.4 Pod to Service ClusterIP

Scenario: Client pod (10.244.134.69) sends HTTP to web-service ClusterIP (10.96.1.X:80).

1. Client pod sends SYN to 10.96.1.X:80.
2. Packet exits pod eth0, enters host namespace.
3. iptables `PREROUTING` table intercepts the packet.
4. Packet matches `KUBE-SERVICES` chain for web-service.
5. `KUBE-SVC-XXX` chain: probability 0.333 → matches `KUBE-SEP-1`.
6. `KUBE-SEP-1` DNAT rule rewrites destination: 10.96.1.X:80 → 10.244.y.y:80 (real pod IP).
7. conntrack records the translation pair.
8. Packet is routed to the real pod (same or different node).
9. Real pod receives SYN, replies with SYN-ACK to source (client IP).
10. Return packet hits iptables `POSTROUTING`. conntrack entry found.

11. Kernel rewrites the source: 10.244.y.y:80 → 10.96.1.X:80.
12. Client pod sees SYN-ACK from the ClusterIP. It never knows the real pod.

Observation commands:

```
CLUSTER_IP=$(k get svc web-service -o jsonpath='{.spec.clusterIP
   }')
WORKER_PID=$(docker inspect k8s-net-lab-worker --format '{{.State.
   Pid}}')

# See the DNAT rules
sudo nsenter -t $WORKER_PID -n iptables -t nat -L -n | grep -A 4 "
   $CLUSTER_IP"

# Capture shows packet twice: ClusterIP dest then real pod IP dest
sudo nsenter -t $WORKER_PID -n tcpdump -i any port 80 -n -c 20 &
k exec client -- curl -s http://$CLUSTER_IP >/dev/null

# See conntrack DNAT pair
sudo nsenter -t $WORKER_PID -n conntrack -L 2>/dev/null | grep "
   $CLUSTER_IP"
```

6.5 Pod DNS Query to CoreDNS

Scenario: Client pod queries DNS for `web-service`.

1. Pod sends UDP DNS query to 10.96.0.10:53 (CoreDNS ClusterIP).
2. Query is: `A? web-service.default.svc.cluster.local` AND `AAAA? web-service.default.svc.clus` simultaneously (`ndots:5` behaviour).
3. iptables DNATs 10.96.0.10:53 to a real CoreDNS pod IP.
4. CoreDNS receives the query. It watches the Kubernetes API and knows web-service's ClusterIP.
5. CoreDNS returns an A record with the web-service ClusterIP.
6. AAAA query returns NODATA (no IPv6 address configured).
7. iptables un-NATtes the response: CoreDNS pod IP → 10.96.0.10.
8. Pod's resolver receives the A record and connects to the ClusterIP.

Observation commands:

```
WORKER_PID=$(docker inspect k8s-net-lab-worker --format '{{.State.
   Pid}}')

# Capture DNS traffic - shows dual A+AAAA queries and DNAT
sudo nsenter -t $WORKER_PID -n tcpdump -i any port 53 -n -c 20 &
k exec client -- nslookup web-service
```

```
# Direct dig with full FQDN
k exec client -- dig web-service.default.svc.cluster.local

# View injected resolv.conf
k exec client -- cat /etc/resolv.conf
```

6.6 Packet Drop Due to NetworkPolicy

Scenario: Frontend pod attempts to send traffic to database pod directly (bypassing backend).

1. Frontend sends SYN to database pod IP.
2. Packet exits frontend pod, enters host namespace on `cali*(frontend)`.
3. Calico's iptables chain `cali-FORWARD` processes the packet.
4. Rule check: source label is `tier=frontend`, destination label is `tier=database`.
5. The NetworkPolicy `allow-backend-to-database` requires source label `tier=backend`. No match.
6. The default rule: DROP. No RST sent.
7. Frontend pod waits for SYN-ACK that never comes. Timeout.

Observation commands:

```
FRONTEND_IP=$(k get pod frontend -o jsonpath='{.status.podIP}')
DATABASE_IP=$(k get pod database -o jsonpath='{.status.podIP}')
WORKER2_PID=$(docker inspect k8s-net-lab-worker2 --format '{{.
   State.Pid}}')

# Attempt the blocked ping -- should time out
k exec frontend -- ping -c 2 $DATABASE_IP || echo "BLOCKED"

# Capture on worker2 -- shows incoming packets but no reply from
   database
sudo nsenter -t $WORKER2_PID -n tcpdump -i any icmp -n -c 10 &
k exec frontend -- ping -c 2 $DATABASE_IP 2>/dev/null

# Show Calico iptables rules (cali-pi = policy ingress rules)
sudo nsenter -t $WORKER2_PID -n iptables -L -n | grep -E "cali|
   DROP"
```

6.7 Cross-Node Health Check via Goldpinger

Scenario: goldpinger-A (on worker) checks goldpinger-B (on worker2).

1. goldpinger-A queries the Kubernetes API for pods with label `app: goldpinger`.

2. The API returns goldpinger-B's pod IP.
3. goldpinger-A sends HTTP GET to goldpinger-B's pod IP:8080.
4. No Service DNAT — Goldpinger communicates via direct pod IPs.
5. The packet follows the cross-node path: cali* → eth0 → Docker bridge → worker2 eth0 → cali* → goldpinger-B.
6. goldpinger-B responds with JSON health data.
7. The result appears in `/check_all`.

Observation commands:

```
GOLDPINGER_IP=$(k get svc goldpinger -o jsonpath='{.spec.clusterIP
   }')

# Query health data
docker exec k8s-net-lab-control-plane curl -s http://
   $GOLDPINGER_IP:8080/check_all \
  | python3 -m json.tool | grep -E '"OK"|response-time'

# Watch live response time between nodes
watch -n 2 'docker exec k8s-net-lab-control-plane curl -s \
  http://'$GOLDPINGER_IP':8080/check_all | python3 -m json.tool |
     grep response-time'
```

6.8 etcd Leader Failure and Raft Recovery

Scenario: etcd-0 (the leader) pod is deleted. Raft elects a new leader and the cluster recovers with zero data loss.

1. `kubectl delete pod etcd-0` executes.
2. etcd-0 sends a `MsgTimeoutNow` to followers (graceful stepdown if possible) before terminating.
3. etcd-1's election timer fires (no heartbeat from leader).
4. etcd-1 increments the term to 3, transitions to Candidate.
5. etcd-1 sends `MsgVote` to etcd-2.
6. etcd-2 grants the vote (it has not voted in term 3 yet).
7. etcd-1 has 2 votes (itself + etcd-2) = quorum. Becomes Leader.
8. etcd-1 sends heartbeats. etcd-2 transitions to Follower.
9. Total elapsed: <4ms.
10. A key written before etcd-0 crashed is readable from etcd-2 (it was already replicated as part of the commit).

11. StatefulSet controller detects etcd-0 is gone and creates a replacement pod.

Observation commands:

```
# Watch election in real time
watch -n 1 'docker exec k8s-net-lab-control-plane kubectl exec
   etcd-1 -- \
  sh -c "ETCDCTL_API=3 etcdctl \
  --endpoints=http://etcd-1.etcd:2379,http://etcd-2.etcd:2379 \
  endpoint status -w table 2>/dev/null"'

# Watch election logs as they happen
docker exec k8s-net-lab-control-plane kubectl logs etcd-1 -f | \
  grep -E "became|elected|election|term|vote"

# Confirm zero data loss after recovery
docker exec k8s-net-lab-control-plane kubectl exec etcd-2 -- sh -c
    \
  "ETCDCTL_API=3 etcdctl --endpoints=http://etcd-2.etcd:2379 get /
     test/phase7"
```

6.9 DNS Failure Triage Decision Tree

The following decision tree is derived directly from Phase 4 and captures the diagnostic logic for Kubernetes service reachability failures:

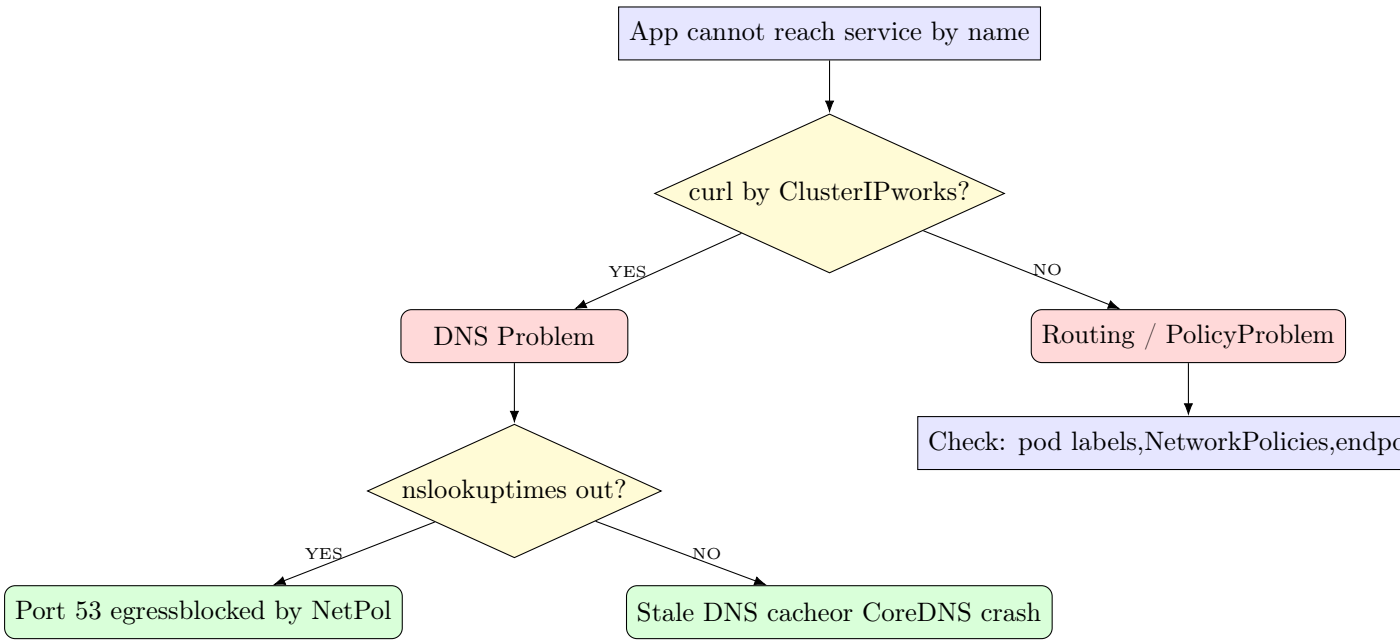

Figure 5: DNS failure triage decision tree (from Phase 4)

7 Control Plane vs Data Plane Deep Dive

This section provides a precise separation of responsibilities for every component in this project, with specific attention to what each component *actually does* versus what another

component handles.

7.1 Separation of Concerns Table

Table 12: Complete component responsibility matrix

Component	Responsibility	Plane
kind	Creates Docker containers as nodes; bootstraps kubeadm; configures networking between containers	Infrastructure
Docker	Provides the container runtime and bridge network for kind nodes	Infrastructure
kube-apiserver	Authenticates requests; validates objects; stores to etcd; notifies watchers	Control
etcd	Persists all Kubernetes object state as key-value pairs	Control
scheduler	Assigns `spec.nodeName` for pending pods based on resources and affinity	Control
controller-manager	Reconciliation loops: ensures desired state = actual state (Deployments, ReplicaSets, etc.)	Control
kubelet	Starts/stops containers via containerd; calls CNI plugin; reports node/pod health to API server	Data
kube-proxy	Programs iptables DNAT chains on every node to implement Service ClusterIP load balancing	Data
Calico	Assigns pod IPs from IPAM; creates veth pairs; programs ip routes; enforces NetworkPolicy via iptables	Data
Linux kernel	Performs actual packet forwarding, NAT rewriting (via netfilter), connection tracking; enforces iptables rules	Data
CoreDNS	Resolves Kubernetes service FQDN to ClusterIP; watches API for service changes; answers DNS queries from pods	Data
Raft	Consensus algorithm inside etcd; governs which node is leader and what constitutes a committed write	Control

7.2 What kind Does (and Doesn't Do)

kind's role ends after cluster creation. It:

- Runs `kubeadm init` inside the control-plane container.

- Runs `kubeadm join` inside each worker container.
- Creates and manages the kubeconfig.
- Configures the Docker bridge networking between nodes.
- Does *not* install a CNI — `disableDefaultCNI: true`.
- Does *not* manage cluster operations after startup.

7.3 What Docker Does

Docker provides the container runtime infrastructure that kind nodes live in. In this project, Docker:

- Creates and manages the `172.18.0.0/16` bridge network.
- Runs the kind node containers.
- Provides `docker exec` for the kubectl alias.
- Provides `docker inspect` for getting node PIDs.
- Does *not* know about Kubernetes pods or services.

7.4 What the Kubernetes Control Plane Does

The control plane is purely a decision-making and state-management system:

- kube-apiserver: serialises all state changes through a single consistent API. Acts as the authoritative source of cluster state.
- etcd: durably stores that state with strong consistency guarantees via Raft.
- scheduler: assigns pods to nodes without moving any packets itself.
- controller-manager: watches for divergence between desired and actual state, and issues corrective actions via the API.

The control plane does not move application packets. If the control plane fails, existing pods continue running and serving traffic. The cluster simply cannot be modified.

7.5 What kubelet Does

kubelet is the bridge between the control plane and the data plane:

1. Watches the API server for pods assigned to its node.
2. Instructs containerd to pull images and create containers.
3. Calls the CNI plugin binary to set up pod networking.
4. Runs liveness and readiness probes.

5. Reports pod status back to the API server.
6. Reports node conditions (Ready, MemoryPressure, etc.) to the API.

kubelet does not forward packets. It sets up the infrastructure that enables packet forwarding.

7.6 What kube-proxy Does

kube-proxy is a reconciliation loop for iptables rules:

1. Watches the API server for Service and Endpoints objects.
2. For each Service, computes the correct probabilistic iptables chains.
3. Atomically replaces old chains with new ones when pods appear or disappear.
4. Does *not* forward packets itself — it only programs the rules.

After kube-proxy programs the rules, it is not in the packet path at all. The Linux kernel iptables subsystem handles packet rewriting independently.

7.7 What Calico Does

Calico operates in three distinct areas:

IPAM Allocates /26 blocks to nodes. Assigns per-pod IPs from those blocks. Stores allocation state in Kubernetes CRDs (Custom Resource Definitions).

Routing Programs Linux `ip route` entries on each node for all pod subnets. Uses BGP for dynamic route distribution (in larger deployments) or static routes.

Policy Translates Kubernetes NetworkPolicy objects into per-pod iptables chains. Updates rules in real-time as pods and policies change.

7.8 What the Linux Kernel Networking Does

The Linux kernel is the actual packet-forwarding engine:

- **ip route**: determines which interface to forward a packet out.
- **netfilter / iptables**: applies PREROUTING DNAT (kube-proxy service translation), FORWARD filtering (Calico NetworkPolicy), and POSTROUTING un-NAT (conntrack-based source IP restoration).
- **conntrack**: tracks the state of all NATed connections. Enables stateful packet inspection and return-path NAT reversal.
- **veth driver**: transfers packets between network namespaces.

7.9 What CoreDNS Does

CoreDNS is an application-layer component:

- Watches the Kubernetes API for Service and pod objects.
- Maintains an in-memory DNS database mapping FQDNs to ClusterIPs.
- Responds to DNS queries from pods.
- For headless services (like the etcd Service in Phase 7), returns individual pod IPs rather than a single VIP.
- Does *not* perform any packet forwarding or routing.

7.10 What etcd Stores

Everything. Every Kubernetes object is serialised and stored in etcd:

```
/registry/pods/default/client
/registry/services/default/web-service
/registry/networkpolicies/default/default-deny-all
/registry/nodes/k8s-net-lab-worker
/registry/deployments/default/web-backend
...
```

The kube-apiserver is the only component that reads/writes etcd directly.

7.11 What Raft Governs

Raft governs *which writes are durable* in the etcd cluster:

- A write is acknowledged to the client only after a quorum of etcd nodes have written it to their logs.
- This prevents data loss if the leader fails after acknowledging a write but before replicating it.
- Raft governs leader election, log replication, term tracking, and membership changes.
- Raft does *not* govern what is stored — only that it is stored durably and consistently.

7.12 Why Packet Captures and conntrack Are Useful for Debugging

Packet captures show you the actual state of the network, not the intended state. They reveal:

- Whether DNAT is actually happening (packet appears twice with different destination IPs).
- Whether traffic is reaching a node at all (versus being dropped before it gets there).
- Whether DNS queries are being sent to the right address.

- The exact TCP state machine (SYN sent but no SYN-ACK = connection refused or dropped).

conntrack reveals the NAT state machine:

- Active connections and their translation pairs.
- Connection states (SYN_SENT, ESTABLISHED, TIME_WAIT).
- Whether a specific ClusterIP is being translated correctly.
- The total number of tracked connections versus the maximum — a leading indicator of conntrack exhaustion.

Together, packet capture and conntrack tell you *what the kernel is actually doing* with packets, independent of what Kubernetes thinks should be happening.

8 Verification and Troubleshooting Lab Guide

This section provides a step-by-step verification guide for every phase of the project. For each command: what it checks, what "good" output looks like, and what broken output suggests.

8.1 Phase 1 Verification

8.1.1 WSL2 and Docker

```
wsl --version            # PowerShell
wsl --list --verbose     # PowerShell      must show VERSION 2
```

Good: Ubuntu is listed with VERSION 2.
Bad: VERSION 1 ⇒ run `wsl -set-version Ubuntu 2`.

```
docker run hello-world
```

Good: "Hello from Docker!" message.
Bad: "Cannot connect to Docker daemon" ⇒ start Docker, or run `sudo service docker start`.

8.1.2 Tool Versions

```
kubectl version --client     # Should show v1.29.x
kind version                 # Should show v0.22.0 or newer
helm version                 # Should show v3.x.x
tcpdump --version            # Must be present for Phase 5
conntrack --version          # Must be present for Phase 5
```

8.1.3 Cluster Creation

```
k get nodes
```

Good:

```
NAME                        STATUS     ROLES           VERSION
k8s-net-lab-control-plane   NotReady   control-plane   v1.29.2
k8s-net-lab-worker          NotReady   <none>          v1.29.2
k8s-net-lab-worker2         NotReady   <none>          v1.29.2
```

Note: `NotReady` is **correct** at this stage.
Bad: Command hangs or TLS timeout ⇒ check the `k` alias is set. Bad: Only 2 nodes appear ⇒ kind creation partially failed; delete and recreate.

8.2 Phase 2 Verification

8.2.1 Calico Installation

```
k get pods -n calico-system
```

Good: All pods show `1/1 Running`:

```
calico-kube-controllers-xxx   1/1 Running
calico-node-xxx               1/1 Running   # One per node
calico-node-xxx               1/1 Running
calico-node-xxx               1/1 Running
calico-typha-xxx              1/1 Running
```

Bad: Any pod in `Pending`, `CrashLoopBackOff`, or `Init:...` ⇒ check `k describe pod <n>` for events.

8.2.2 Node Readiness

```
k get nodes
```

Good: All three nodes show `Ready`.
Bad: Still `NotReady` after 3+ minutes ⇒ Calico is not fully healthy. Check calico-node DaemonSet logs.

8.2.3 Routing Table

```
docker exec k8s-net-lab-control-plane ip route | grep "proto 80"
```

Good: Entries for each worker's pod subnet with `via 172.18.0.x`:

```
10.244.11.0/26 via 172.18.0.7 dev eth0 proto 80   # worker2 pods
10.244.134.64/26 via 172.18.0.6 dev eth0 proto 80 # worker pods
```

Bad: No `proto 80` routes ⇒ Calico did not successfully program routes. Check calico-node pod logs.

8.2.4 Cross-Node Ping

```
POD_B_IP=$(k get pod pod-b -o jsonpath='{.status.podIP}')
k exec pod-a -- ping -c 4 $POD_B_IP
```

Good: 0% packet loss, `ttl=62` (proving cross-node routing).
Bad: 100% loss ⇒ routing is broken. Check routes, calico-node status, and whether CNI is applying policy that blocks ICMP.

8.2.5 veth Interfaces

```
docker exec k8s-net-lab-worker ip link show type veth | grep cali
```

Good: One `cali*` interface per pod running on that node.
Bad: No `cali*` interfaces ⇒ Calico node agent is not creating veth pairs. Restart calico-node DaemonSet.

8.3 Phase 3 Verification

8.3.1 Default-Deny Working

```
k exec pod-a -- ping -c 2 <pod-b-ip> 2>&1 | grep -E "loss|BLOCKED"
```

Good: 100% packet loss (no output from ping = traffic silently dropped).
Bad: Pings succeed ⇒ NetworkPolicy was not applied, or Calico is not enforcing policy. Check `k get networkpolicies`.

8.3.2 3-Tier Policy Matrix

Run all four test commands and verify:

```
k exec frontend -- ping -c 2 $BACKEND_IP  # MUST SUCCEED
k exec backend  -- ping -c 2 $DATABASE_IP # MUST SUCCEED
k exec frontend -- ping -c 2 $DATABASE_IP # MUST FAIL
k exec database -- ping -c 2 $BACKEND_IP  # MUST FAIL
```

Bad: A path that should be blocked succeeds ⇒ Check `k get pods -show-labels` vs `k describe networkpolicy`. The `podSelector` likely references a label that doesn't exist on the pod (Issue 7).

8.4 Phase 4 Verification

8.4.1 Service ClusterIP

```
k get service web-service
```

Good: `TYPE: ClusterIP`, a valid IP in the `10.96.0.0/12` range, `PORT(S): 80/TCP`.

8.4.2 Endpoints Populated

```
k get endpoints web-service
```

Good: 3 pod IPs listed under ENDPOINTS.
Bad: Empty or `<none>` ⇒ Pods are not `Ready` or the Service `selector` does not match pod labels.

8.4.3 iptables DNAT Rules

```
WORKER_PID=$(docker inspect k8s-net-lab-worker --format '{{.State.
   Pid}}')
CLUSTER_IP=$(k get svc web-service -o jsonpath='{.spec.clusterIP
   }')
sudo nsenter -t $WORKER_PID -n iptables -t nat -L -n | grep -A 3 "
   $CLUSTER_IP"
```

Good: `KUBE-SVC` entry for the ClusterIP, followed by `KUBE-SEP` rules with probability values (0.333, 0.5, 1.0 for 3 pods).
Bad: No rules for the ClusterIP ⇒ kube-proxy has not programmed rules. Check kube-proxy pod: `k get pods -n kube-system`.

8.4.4 DNS Resolution

```
k exec client -- nslookup web-service
```

Good: Resolves to the ClusterIP, answered by `10.96.0.10`.
Bad: Timeout ⇒ Check if CoreDNS is running (`k get pods -n kube-system | grep coredns`).

8.4.5 DNS Failure Signature Test

```
# After applying block-dns NetworkPolicy:
k exec client -- nslookup web-service           # MUST FAIL (
   timeout)
k exec client -- curl --max-time 3 http://$CLUSTER_IP  # MUST
   SUCCEED
```

Good: Name fails, IP succeeds — confirms DNS is the only broken layer.

8.5 Phase 5 Verification

8.5.1 nsenter Works

```
NODE_PID=$(docker inspect k8s-net-lab-worker --format '{{.State.
   Pid}}')
echo "PID: $NODE_PID"
sudo nsenter -t $NODE_PID -n ip link   # Should list node
   interfaces
```

Good: Lists `eth0`, `cali*` interfaces.
Bad: `nsenter: failed to open...` ⇒ Run with `sudo`; check PID is valid.

8.5.2 DNAT Visible in tcpdump

```
NODE_PID=$(docker inspect k8s-net-lab-worker --format '{{.State.
   Pid}}')
sudo nsenter -t $NODE_PID -n tcpdump -i any port 80 -n -c 10 &
k exec client -- curl -s http://$CLUSTER_IP > /dev/null
wait
```

Good: Packet appears on `cali*` with ClusterIP destination, then appears on `eth0` with real pod IP destination — proving DNAT.
Bad: No packets at all ⇒ traffic is not reaching this node (may be on the other worker node).

8.5.3 conntrack DNAT Pairs

```
sudo nsenter -t $NODE_PID -n conntrack -L 2>/dev/null | grep "
   $CLUSTER_IP" | head -5
```

Good: Entries showing original (ClusterIP) and translated (pod IP) tuples.

8.6 Phase 6 Verification

8.6.1 Goldpinger Pods Running

```
k get pods -l app=goldpinger -o wide
```

Good: Two pods, one per worker node, both `1/1 Running`.

8.6.2 Cross-Node Health

```
GOLDPINGER_IP=$(k get svc goldpinger -o jsonpath='{.spec.clusterIP
   }')
docker exec k8s-net-lab-control-plane curl -s http://
   $GOLDPINGER_IP:8080/check_all \
  | python3 -m json.tool | grep '"OK"'
```

Good: All `"OK": true`.
Bad: `"OK": false` ⇒ cross-node pod communication is broken. Check Calico routes.
Bad: `"hosts": null` ⇒ RBAC is broken. Check ServiceAccount and ClusterRoleBinding.

8.7 Phase 7 Verification

8.7.1 etcd Cluster Healthy

```
docker exec k8s-net-lab-control-plane kubectl exec etcd-0 -- sh -c
    \
  "ETCDCTL_API=3 etcdctl \
  --endpoints=http://etcd-0.etcd:2379,http://etcd-1.etcd:2379,http
     ://etcd-2.etcd:2379 \
  endpoint status -w table"
```

Good: All 3 endpoints listed, one with `IS LEADER: true`, all at the same `RAFT INDEX`.
Bad: One endpoint shows timeout ⇒ Check pod status (`k get pods | grep etcd`).
Bad: Crash loop ⇒ Bootstrap conflict (Issue 8). Full redeploy required.

8.7.2 Raft Write and Read

```
# Write to leader
k exec etcd-0 -- sh -c "ETCDCTL_API=3 etcdctl \
  --endpoints=http://etcd-0.etcd:2379 put /test/verify 'working'"

# Read from follower
k exec etcd-2 -- sh -c "ETCDCTL_API=3 etcdctl \
  --endpoints=http://etcd-2.etcd:2379 get /test/verify"
```

Good: `working` returned from etcd-2.
Bad: Empty or timeout ⇒ Replication is not working. Check that all peers are in the same term.

8.7.3 Failure Mode 1 — Leader Loss

```
# Trigger
k delete pod etcd-0

# Observe (within 10 seconds)
k get pods | grep etcd           # etcd-0 Terminating, then new
   pod starting
k logs etcd-1 | grep "became leader"  # New leader logged

# Verify zero data loss
k exec etcd-1 -- sh -c "ETCDCTL_API=3 etcdctl \
  --endpoints=http://etcd-1.etcd:2379 get /test/phase7"
```

Good: `raft-is-working` returned from the new leader.

8.8 Quick Diagnostics Cheat Sheet

```
# Cluster reachable?
k get nodes

# All pods healthy?
k get pods -A

# Pod stuck? Why?
k describe pod <pod-name>
k logs <pod-name> --tail=20
k logs <pod-name> --previous     # Logs from last crash

# Service not routing traffic?
k get endpoints <service-name>   # NotReadyAddresses = pod not
   Ready
```

```
# NetworkPolicy matching?
k get pods --show-labels          # Check actual labels
k describe networkpolicy <name>   # Check PodSelector

# DNS working?
k exec <pod> -- nslookup kubernetes
k exec <pod> -- cat /etc/resolv.conf

# Traffic flowing?
NODE_PID=$(docker inspect k8s-net-lab-worker --format '{{.State.
   Pid}}')
sudo nsenter -t $NODE_PID -n tcpdump -i any -n -c 20

# conntrack exhaustion risk?
sudo nsenter -t $NODE_PID -n conntrack -C 2>/dev/null
```

9 Weaknesses, Oddities, and Cleanup Opportunities

This section critically evaluates the repository from the perspective of a senior engineer reviewing it for professionalism, reproducibility, and accuracy.

9.1 WSL2-Only Assumptions

The entire project is built around a WSL2 environment. This creates several assumptions that are not portable:

- The `k` alias (`docker exec ...`) only works when Docker is accessible from a Unix shell. On a native Linux machine, standard `kubectl` with kubeconfig would work without the alias.
- The `docker inspect -format '.State.Pid'` pattern for nsenter is specific to kind + Docker environments. On a cloud-based cluster, there are no Docker containers to inspect.
- Port 6443 TLS timeout is a WSL2-specific behaviour. On Linux, `kubectl` would work directly.

Cleanup opportunity: Add a *native Linux variant* note in the README explaining that on a real Linux machine (not WSL2), the `k` alias is not needed and standard kubeconfig-based `kubectl` works.

9.2 Environment-Specific Workarounds

Table 13: Workarounds in this project and their limitations

Workaround	Why Needed	Production Alternative
`k` alias via `docker exec`	WSL2 bridge not routable from userspace	Standard kubeconfig / `kubectl`
`docker cp` before applying manifests	docker exec doesn't pass stdin	Use a cluster reachable by kubeconfig
SHA-pinned kind node image	WSL2 cgroup incompatibility with new images	Use a compatible kernel / native Linux
nsenter for tcpdump	kind nodes have no diagnostic tools	Sidecar debug containers; `kubectl debug`
No PVCs on etcd StatefulSet	Local kind cluster has no dynamic provisioner	Always use PVCs in production

9.3 Fragile Steps

1. **Kind version mismatch:** Phase 1 installs kind v0.22.0 but the README states v0.31.0. This discrepancy means the cluster config and images may or may not match depending on which version was actually installed.

2. **ContainerLab conflict not obvious:** Issue 3 (ContainerLab network conflicts) is documented in `issues.md` but not mentioned in the relevant phase docs. An engineer running ContainerLab alongside kind will hit this failure and may not know to look in issues.md.

3. **CoreDNS crash-loop dependency:** CoreDNS will crash-loop if applied before Calico is fully healthy (Issue 5). The Phase 2 docs say to wait until calico-system pods are `1/1 Running` but don't explain *why* the order matters.

4. **etcd no-PVC design:** The etcd StatefulSet intentionally omits PVCs to enable Failure Mode 3. This is educationally correct but creates a fragile setup where any pod restart requires a full redeploy.

9.4 Manual Steps That Could Be Automated

- The `k` alias setup could be in a `setup.sh` script that also installs all tools and creates the cluster in one command.

- The `docker cp` pattern for each manifest is repetitive. A helper function `kapply()` could abstract it:

```
kapply() {
  docker cp "$1" k8s-net-lab-control-plane:/"$1"
```

```
  docker exec k8s-net-lab-control-plane kubectl apply -f /"$1"
}
```

- IP variable extraction (`POD_B_IP=$(k get pod ...)`) is repeated in nearly every phase. A `get_pod_ip` helper would reduce repetition.
- Phase 7 failure mode setup (isolation NetworkPolicy + watch command) could be a single script with automated output capture.

9.5 Places Where Docs May Be Incomplete or Misleading

- **Phase 2, iptables inspection:** Step 6 shows how to view Calico iptables rules but does not explain the chain naming convention (`cali-FORWARD`, `cali-pi-in-xxx`) — a gap for readers trying to understand what they see.
- **Phase 6, eBPF section:** The section acknowledges that Cilium was not deployed on WSL2 and covers eBPF conceptually. This is honest and appropriate, but a reader might expect to run the eBPF commands and be disappointed.
- **Phase 7, election timing:** The "<4ms recovery" claim is accurate for the specific conditions of this lab (all nodes on the same Docker bridge, no real network latency). In production, election timeout is typically 150–300ms by default. The docs should note this.
- **README version table:** Lists Calico v3.27.0 and kubectl v1.35.3 (the latter is likely a typo; v1.35.x does not exist as of writing).

9.6 What Could Be Improved for Professionalism

1. **A single `setup.sh`** that installs all tools, creates the cluster, installs Calico, and runs Phase 1–2 validation automatically. This would demonstrate infrastructure-as-code thinking.
2. **A `validate.sh` per phase** that runs all verification commands and prints pass/fail for each. This would make the lab reproducible as a CI test.
3. **Manifests in a `manifests/` directory** rather than heredoc-defined inline. Separating manifests from runbook text would make them reusable and version-controllable.
4. **A diagram file** (Mermaid or draw.io) showing the full cluster architecture. Currently diagrams exist only in prose.
5. **Consistent tool versions** between phase docs and the README.

9.7 What Is Excellent

Despite the above, several aspects of this project are genuinely impressive:

- **issues.md** is exceptional. Ten fully documented issues with exact error text, root cause analysis, and working fixes demonstrate real debugging expertise. This alone is strong interview material.

- **TTL evidence for cross-node routing** (`ttl=62`) is a subtle but technically precise proof. Many Kubernetes tutorials show ping succeeding without understanding why TTL decrements.
- **DNS failure signature documentation** is production-grade triage knowledge. The "name fails, IP works" pattern is not well documented in most learning resources.
- **Raft recovery log evidence** with actual term numbers and election timing is unusually thorough for a lab project.
- **Explicit acknowledgement of WSL2 limitations** rather than presenting workarounds as the correct approach.
- **Goldpinger partition simulation** (Step 5, Phase 6) turns an observability tool into a network testing tool — a creative and technically sound use.

10 Interview-Ready Understanding

This section prepares you to discuss this project confidently in interviews for network engineering, platform engineering, and SRE roles. It provides three levels of response, likely interview questions with strong and weak answers, and identifies the most technically impressive parts of the project.

10.1 Elevator-Pitch Levels

10.1.1 30-Second Version (Recruiter Screen)

> *"I built a seven-phase Kubernetes networking lab on WSL2 using kind to simulate a three-node cluster inside Docker containers. I layered in Calico CNI for pod networking, built a zero-trust three-tier application with NetworkPolicy, deployed ClusterIP Services, and traced DNS through CoreDNS. In Phase 5 I did live packet capture via nsenter to verify DNAT translation and conntrack state pairs without installing any tools on the nodes. I finished by deploying a standalone three-node etcd Raft cluster and deliberately causing three documented failure modes — leader loss, DNS exclusion of a NotReady pod, and a bootstrap conflict — with verified recovery procedures. The entire project runs in WSL2, so it works on a Windows laptop with no cloud spend."*

10.1.2 2-Minute Version (Hiring Manager)

> I used kind to create three Kubernetes nodes as Docker containers on a single WSL2 machine. After Phase 1 the nodes were deliberately left in NotReady state because I had disabled the default CNI — this makes the CNI-to-Ready transition visible and learnable.
>
> Phase 2 installed Calico. I watched nodes flip from NotReady to Ready in real time, then inspected the routing table and found Calico-programmed routes marked proto 80 pointing each worker's /26 pod subnet to the correct Docker

bridge next-hop. I captured a cross-node ping and confirmed ttl=62, proving the packet transited two nodes and decremented twice.

Phase 3 implemented zero-trust with a default-deny NetworkPolicy on all pods, then added selective allows for a frontend→backend→database flow. I ran a full four-path test matrix — two allowed, two blocked — and verified with nsenter that Calico had programmed the corresponding iptables chains.

Phase 4 showed that a ClusterIP is a virtual IP that exists only in kube-proxy iptables DNAT rules. I read the KUBE-SVC and KUBE-SEP chains and saw the 0.333/0.5/1.0 probabilistic load-balancing structure for three replicas. I then reproduced the DNS failure signature — blocking port 53 egress makes the service unreachable by name while the ClusterIP still works directly.

Phase 5 used nsenter to enter node network namespaces from the WSL2 host and run tcpdump without any tools installed on the nodes. The packet capture showed DNAT translation live: the same HTTP SYN appears on the cali* interface with the ClusterIP destination, then on eth0 with the real pod IP after DNAT. conntrack entries confirmed the original and translated tuple pair.

Phase 6 deployed Goldpinger as a DaemonSet for continuous cross-node health. I partitioned one pod with a NetworkPolicy and watched the health check immediately reflect the failure.

Phase 7 deployed a standalone three-node etcd Raft cluster as a StatefulSet inside the kind cluster. I documented three failure modes: leader election in under four milliseconds with zero data loss, correct DNS exclusion of a NotReady pod, and a bootstrap conflict from a missing PVC that cascades into a CrashLoopBackOff. I also hit ten significant environment issues during the project — every one is in issues.md with exact root cause and working fix."*"The core motivation was to understand what actually happens at the Linux kernel level when Kubernetes moves a packet — not just 'apply and hope.'*

I used kind to create three Kubernetes nodes as Docker containers on a single WSL2 machine. After Phase 1 the nodes were deliberately left in NotReady state because I had disabled the default CNI — this makes the CNI-to-Ready transition visible and learnable.

Phase 2 installed Calico. I watched nodes flip from NotReady to Ready in real time, then inspected the routing table and found Calico-programmed routes marked proto 80 pointing each worker's /26 pod subnet to the correct Docker bridge next-hop. I captured a cross-node ping and confirmed ttl=62, proving the packet transited two nodes and decremented twice.

Phase 3 implemented zero-trust with a default-deny NetworkPolicy on all pods, then added selective allows for a frontend→backend→database flow. I ran a full four-path test matrix — two allowed, two blocked — and verified with nsenter that Calico had programmed the corresponding iptables chains.

Phase 4 showed that a ClusterIP is a virtual IP that exists only in kube-proxy iptables DNAT rules. I read the KUBE-SVC and KUBE-SEP chains and saw the 0.333/0.5/1.0 probabilistic load-balancing structure for three replicas. I

then reproduced the DNS failure signature — blocking port 53 egress makes the service unreachable by name while the ClusterIP still works directly.

Phase 5 used nsenter to enter node network namespaces from the WSL2 host and run tcpdump without any tools installed on the nodes. The packet capture showed DNAT translation live: the same HTTP SYN appears on the cali interface with the ClusterIP destination, then on eth0 with the real pod IP after DNAT. conntrack entries confirmed the original and translated tuple pair.*

Phase 6 deployed Goldpinger as a DaemonSet for continuous cross-node health. I partitioned one pod with a NetworkPolicy and watched the health check immediately reflect the failure.

Phase 7 deployed a standalone three-node etcd Raft cluster as a StatefulSet inside the kind cluster. I documented three failure modes: leader election in under four milliseconds with zero data loss, correct DNS exclusion of a NotReady pod, and a bootstrap conflict from a missing PVC that cascades into a CrashLoopBackOff. I also hit ten significant environment issues during the project — every one is in issues.md with exact root cause and working fix."

10.1.3 Deep Technical Version (Senior/Staff Engineer)

The sections below provide the detailed answers your interviewers want. Internalise these before a senior network engineer, platform engineer, or SRE interview.

10.2 Topic-by-Topic Deep Technical Answers

10.2.1 Calico CNI and Pod Routing

11 Learning Roadmap

This roadmap is designed specifically for a CCNA-certified engineer who has completed this lab and wants to build toward cloud-native network engineering, platform engineering, or SRE roles. Topics are grouped into four tiers based on priority and dependency order.

11.1 Tier 1 — Master Now (Weeks 1–4)

These concepts are directly demonstrated in this lab and are the highest probability interview topics. Study them until you can explain every mechanism without notes.

Table 14: Tier 1: Core concepts to internalise from this lab

Concept	What to master	Time estimate
CNI and pod lifecycle	Why nodes are NotReady without CNI. What kubelet does to call the CNI plugin. veth pair creation. ip route host routes. blackhole route purpose.	2 days
Calico IPAM and routing	/26 per-node block allocation. proto 80 route marker. Cross-node via Docker/next-hop. VXLANCrossSubnet concept. ttl=62 as proof.	2 days
kube-proxy DNAT	iptables nat table. KUBE-SVC / KUBE-SEP chain hierarchy. Probabilistic load-balancing maths. ClusterIP as virtual IP. conntrack translation pair. Return path un-NAT.	2 days
CoreDNS and DNS failure	FQDN format. ndots:5 behaviour. Dual A+AAAA queries. DNS failure signature. Triage tree (name fails, IP works). Blocking port 53 via NetworkPolicy.	1 day
NetworkPolicy	Default-deny-all foundation. Additive allow model. Label-based selectors. Silent drop behaviour. Ingress vs egress. Label mismatch silent failure. iptables enforcement by Calico.	2 days
nsenter + tcpdump	Getting node PID via docker inspect. `nsenter -t $PID -n` pattern. Interface selection. DNAT visible as double-packet. Filtering by port.	1 day
conntrack	Original/translated tuple structure. TIME_WAIT, ESTABLISHED states. Exhaustion at nf_conntrack_max. Reading count with conntrack -C.	1 day
Raft basics	Leader, follower, candidate roles. Quorum = majority. Term as election epoch. Log replication before commit. Leader loss and recovery sequence. Data loss only for uncommitted writes.	2 days
etcd operational basics	Client port 2379, peer port 2380. etcdctl endpoint status output. IS LEADER and RAFT TERM fields. Reading and writing keys.	1 day

11.1.1 How to Study Tier 1

1. **Rebuild from scratch.** Delete the kind cluster and reproduce all seven phases from memory, using the phase docs only when stuck. The act of rebuilding makes the concepts stick.
2. **Explain each step aloud.** Before running a command, say what you expect to see and why. After seeing the output, confirm or revise your prediction.
3. **Break things deliberately.** Misconfigure the Calico cidr to not match kind-cluster.yaml and observe the failure. Delete a kube-proxy pod and watch service routing degrade. These deliberate failures teach you the failure modes you will encounter in production.
4. **Read the corresponding docs sections** from Section 12 for each concept while the lab is running. Cross-referencing theory with live evidence accelerates retention.

11.2 Tier 2 — Study Soon (Months 1–3)

These concepts build on Tier 1 and are critical for senior/staff engineering roles. They also appear in CKA certification material.

Table 15: Tier 2: Intermediate concepts to study after completing the lab

Concept	Why it matters and where to start	Priority
eBPF fundamentals	Cilium is replacing kube-proxy and Calico iptables in modern clusters. eBPF programs run in-kernel at wire speed without rule-chain traversal. Study the Cilium docs and the eBPF.io learning materials.	High
Calico BGP mode	Production large-scale clusters use BGP peering between Calico nodes rather than VXLAN. Understand AS numbers, peer configurations, and route propagation. CCNA BGP knowledge applies directly.	High
Ingress Controllers	How external traffic enters a Kubernetes cluster. nginx, Traefik, and Contour are common. Study how an Ingress object maps to a Service, and how TLS termination works.	High
Kubernetes RBAC	ServiceAccounts, ClusterRoles, RoleBindings, ClusterRoleBindings. Required for any workload that calls the Kubernetes API (like Goldpinger). Appears in virtually every production cluster.	High
PersistentVolumes and PVCs	Critical for stateful workloads. Study StorageClass, dynamic provisioning, and ReadWriteOnce vs ReadWriteMany. Directly explains the Failure Mode 3 fix.	Medium
Prometheus + Grafana	Standard Kubernetes observability stack. Goldpinger exposes Prometheus metrics. Understand the scrape model, metric types, and how to build a network health dashboard from Goldpinger data.	Medium
Service Mesh basics (Istio / Linkerd)	Extends NetworkPolicy to Layer 7: mTLS, traffic management, retries, circuit breakers, distributed tracing. Istio is used at most large organisations. Study the sidecar proxy model and how it interacts with existing NetworkPolicy.	Medium
IPVS mode kube-proxy	IPVS uses kernel hash tables instead of iptables chains, giving O(1) rule lookup regardless of service count. Study ipvsadm command output and compare with the iptables chains you already know.	Medium
kubectl debug and ephemeral containers	The officially supported alternative to nsenter for production clusters where you cannot use docker inspect.	Medium

11.2.1 Tier 2 Lab Extensions

1. **Add PVCs to the etcd StatefulSet.** Modify Phase 7 to use `volumeClaimTemplates` and verify that restarting `etcd-0` does not cause the bootstrap conflict.
2. **Switch kube-proxy to IPVS mode.** Edit the kube-proxy ConfigMap to use `mode: ipvs`, restart kube-proxy pods, and compare `ipvsadm -ln` output with the iptables chains you captured before.
3. **Deploy nginx Ingress.** Install the nginx ingress controller via Helm, create an Ingress resource pointing to `web-service`, and trace the path from external HTTP to the pod.
4. **Add Prometheus and Goldpinger metrics.** Scrape Goldpinger's `/metrics` endpoint with Prometheus and build a Grafana panel showing per-node pair latency over time.

11.3 Tier 3 — Revisit Later (Months 3–6+)

These are important for staff/principal engineers and architects. They require Tier 1 and Tier 2 as solid prerequisites.

Table 16: Tier 3: Advanced concepts for future study

Concept	Context and relevance
etcd operational management	Compaction, defragmentation, TLS certificate rotation, backup and restore procedures, watch event management, and performance tuning. Critical for anyone running a self-managed Kubernetes cluster.
Raft edge cases	Pre-vote extension (prevents disruptive elections from partitioned nodes), learner nodes, joint consensus for membership changes, linearizability vs serializability, leader lease. Relevant for anyone building or operating distributed databases.
eBPF programming	Writing custom eBPF programs using libbpf or the cilium/ebpf Go library. Custom traffic classifiers, socket-level observability, XDP packet processing. Emerging skill for senior network engineers.
Multi-cluster networking	Cilium ClusterMesh, Submariner, Skupper, and cloud-native VPN overlays for connecting multiple Kubernetes clusters. Relevant for multi-region deployments.
CNI plugin development	Implementing the CNI spec from scratch. Deepens understanding of the plugin contract between kubelet and CNI, and of how IP routing and namespaces work at the syscall level.
Kubernetes networking at scale	conntrack table tuning (nf_conntrack_max, nf_conntrack_buckets), IPVS vs iptables performance benchmarks at 10k+ services, kube-proxy synchronisation latency, and CNI choice at large scale.
Service Mesh advanced	Envoy xDS API, certificate rotation with cert-manager, mutual TLS enforcement via PeerAuthentication, traffic policy CRDs in Istio. Required for zero-trust at L7.

11.4 What to Ignore Initially

The following Kubernetes topics are important in broader engineering contexts but are not related to networking and should be deferred until your networking foundation is solid:

- Kubernetes operator development and CRD schema design
- CI/CD pipeline tooling (ArgoCD, Flux, Tekton, GitHub Actions)
- Container image building, multi-stage Dockerfiles, image scanning
- Kubernetes API extension (admission webhooks, aggregated API servers)

- Multi-tenancy patterns, namespace isolation, hierarchical namespaces
- Cost optimisation, node autoscaling, resource requests and limits tuning
- Kubernetes-native secret management and external secret operators

These are all valid and important topics. They just do not build on this lab's networking focus, and studying them before mastering Tier 1 will dilute your retention.

11.5 Lab Practice Methodology

11.5.1 The Three-Phase Practice Cycle

For each concept in Tier 1:

1. **Reproduce it** — run the lab commands, observe the output.
2. **Break it** — deliberately introduce a failure and debug it.
3. **Explain it** — describe what you did, why it works, and what the failure mode is, as if presenting to a colleague.

11.5.2 Suggested Additional Lab Exercises

1. **5-node etcd cluster and 2-node failure.** Quorum = 3. Kill 2 nodes: cluster continues. Kill a 3rd: writes stop. Verify that reads still succeed after quorum loss.
2. **Manual IP route manipulation.** Delete a Calico-programmed cross-node route (`ip route del` on the node) and observe what happens to cross-node pod communication. Watch Calico reprogram it within seconds.
3. **conntrack exhaustion simulation.** Reduce `nf_conntrack_max` to a small value via sysctl in a node namespace and observe connections being silently dropped at that threshold.
4. **coreDNS failure injection.** Scale CoreDNS Deployment to 0 replicas and observe the cascade: all service-name-based access fails, ClusterIP-based access continues.
5. **Rebuild without the k alias.** On a native Linux VM (not WSL2), reproduce Phase 1 using standard kubeconfig-based `kubectl` to understand why the WSL2 workaround exists and what the normal path looks like.

11.6 Certification and Career Path Alignment

Table 17: This lab aligned to industry certifications

Certification	Lab overlap	Gap to fill	Priority
CKA (Kubernetes Administrator)	High: cluster provisioning, CNI, Services, DNS, RBAC (Goldpinger), NetworkPolicy, etcd backup concepts, troubleshooting	etcd backup/restore, TLS cert rotation, cluster upgrades, multi-node drain	Highest
CKAD (Kubernetes Developer)	Medium: pods, deployments, services, DNS lookup patterns	Multi-container patterns, ConfigMaps, Secrets, Jobs, CronJobs, resource limits	Medium
CKS (Kubernetes Security Specialist)	High: NetworkPolicy zero-trust, RBAC, pod security contexts (conceptual)	Pod Security Admission, supply chain security, runtime threat detection, Falco	High (after CKA)
AWS/GCP/Azure Kubernetes certs	Medium: networking concepts transfer, but cloud-specific load balancers, IAM integration, and managed CNI differ	Cloud-specific: VPC CNI (AWS), Calico on cloud, managed etcd, node pools	After CKA

Note

Recommended path: Study this lab → pursue CKA (the lab covers ≈60% of the CKA syllabus at deeper-than-exam depth) → pursue CKS → add cloud provider networking specialisation (AWS VPC CNI, GKE Dataplane V2 which is Cilium, AKS CNI modes).

11.7 Estimated Time Investment

Table 18: Estimated total study and lab time

Activity	Hours	When	Output
Run this lab (Phases 1–7)	25–35	Week 1–2	Working cluster, documented
Re-run from memory	8–12	Week 3	Retained understanding
Tier 1 reading + docs	15–20	Week 2–4	Concept depth
Tier 2 lab extensions	20–30	Month 2	Broader capability
CKA preparation	40–60	Month 2–3	Certification
Tier 3 exploration	40–80	Month 4–6	Advanced depth
Total to CKA	**110–160**	**3 months**	**Job-ready**

12 Reference Library

All references are categorised by topic area. Each entry includes the URL, a note on why it is useful in the context of this lab, and an experience level tag (Beginner / Intermediate / Advanced).

12.1 Kubernetes Fundamentals

Kubernetes Concepts [32]
https://kubernetes.io/docs/concepts/
The primary starting point. Covers architecture, workloads, services, storage, configuration, and security. Every concept in this lab has a corresponding page here. *Level: Beginner–Intermediate*

Kubernetes: The Hard Way [18]
https://github.com/kelseyhightower/kubernetes-the-hard-way
Builds a cluster manually without kubeadm, exposing every certificate, binary, and configuration step that kubeadm and kind automate. The single best resource for deep understanding of cluster internals. *Level: Intermediate*

How a Kubernetes Pod Gets Its IP Address [38]
https://learnk8s.io/kubernetes-network-packets
Highly detailed walkthrough of the full packet path from pod to pod, with diagrams at every hop. Directly complementary to Phases 2–5 of this lab. *Level: Intermediate*

Understanding Kubernetes Networking Model [47]
https://sookocheff.com/post/kubernetes/understanding-kubernetes-networking-model
Clear explanation of the Kubernetes network model and the requirements it places on CNI implementations. Good prerequisite reading for Phase 2. *Level: Intermediate*

12.2 kind — Kubernetes IN Docker

kind Quick Start [24]
https://kind.sigs.k8s.io/docs/user/quick-start/
Official documentation for cluster creation, configuration options, and the multi-node cluster setup used in Phase 1. *Level: Beginner*

kind Configuration Reference [22]
https://kind.sigs.k8s.io/docs/user/configuration/
All cluster configuration options: `disableDefaultCNI`, `podSubnet`, `serviceSubnet`, node image pinning, port mappings. *Level: Intermediate*

kind Known Issues [23]
https://kind.sigs.k8s.io/docs/user/known-issues/
Documents the WSL2-specific networking limitations that are the root cause of Issues 1–3 in this project. *Level: Beginner*

kind Using WSL2 [25]
https://kind.sigs.k8s.io/docs/user/using-wsl2/
kind's official WSL2 guidance. Explains why the docker exec alias is the recommended kubectl access pattern on WSL2. *Level: Beginner*

12.3 kubectl

kubectl Cheat Sheet [30]
https://kubernetes.io/docs/reference/kubectl/cheatsheet/
Quick reference for all `kubectl` commands used in this lab. Keep this open while working through the phases. *Level: Beginner*

Organising Cluster Access with kubeconfig [36]
https://kubernetes.io/docs/concepts/configuration/organize-cluster-access-kubeco
Explains the kubeconfig structure, contexts, and why stale kubeconfig causes Issue 9 in this lab. *Level: Beginner*

kubectl Reference Docs [31]
https://kubernetes.io/docs/reference/generated/kubectl/kubectl-commands
Complete reference for every kubectl subcommand including all flags. *Level: Beginner–Intermediate*

12.4 Helm

Helm Documentation [17]
https://helm.sh/docs/
Official Helm documentation. Chart structure, template syntax, values, releases, and rollbacks. Useful for understanding how Goldpinger or Calico could be installed via Helm. *Level: Beginner–Intermediate*

Helm Chart Best Practices [16]
https://helm.sh/docs/chart_best_practices/
Production patterns for chart design: labels, templates, values, and dependency management. *Level: Intermediate*

12.5 CNI Basics

CNI Specification [9]
https://github.com/containernetworking/cni/blob/main/SPEC.md
The actual CNI specification document. Defines the ADD/DEL command contract between kubelet and CNI plugins. Explains what Calico must implement when called by kubelet. *Level: Intermediate*

Kubernetes Network Model [26]
https://kubernetes.io/docs/concepts/cluster-administration/networking/
Explains the three requirements Kubernetes places on any CNI: pods must communicate without NAT, nodes must communicate with pods without NAT, and every pod gets a routable IP. *Level: Beginner*

CNI Reference Plugins [8]
https://github.com/containernetworking/plugins
Reference implementation of CNI plugins including bridge, host-local IPAM, and loopback. Useful for understanding the CNI contract at the code level. *Level: Advanced*

12.6 Calico

Calico Networking Documentation [52]
https://docs.tigera.io/calico/latest/networking/
Official documentation for Calico IP pools, BGP configuration, VXLAN encapsulation, and IPAM. Directly maps to Phase 2 of this lab. *Level: Intermediate*

Calico IPAM [50]
https://docs.tigera.io/calico/latest/networking/ipam/
Explains blockSize, per-node subnet allocation, and the IP pool design used in this lab (/26 per node from 10.244.0.0/16). *Level: Intermediate*

Calico NetworkPolicy [51]
https://docs.tigera.io/calico/latest/network-policy/
How Calico translates Kubernetes NetworkPolicy objects into iptables or eBPF rules. Explains the enforcement model used in Phase 3. *Level: Intermediate–Advanced*

Tigera Operator Installation [53]
https://docs.tigera.io/calico/latest/operations/operator-based-installation
Explains the operator-based installation used in Phase 2 and the role of the Installation custom resource. *Level: Beginner*

Calico Architecture Overview [49]
https://docs.tigera.io/calico/latest/reference/architecture/overview
Component breakdown: calico-node, calico-kube-controllers, calico-typha. Maps to the `k get pods -n calico-system` output. *Level: Intermediate*

12.7 Kubernetes Services and kube-proxy

Kubernetes Services [37]

https://kubernetes.io/docs/concepts/services-networking/service/
Official documentation for ClusterIP, NodePort, LoadBalancer, headless services, and the iptables proxy mode. Authoritative reference for everything in Phase 4. *Level: Beginner–Intermediate*

Cracking the Kubernetes Node Proxy [6]
https://arthurchiao.art/blog/cracking-k8s-node-proxy/
The most thorough independent analysis of kube-proxy iptables mode. Walks through every iptables chain by name and explains the probability calculations. Required reading for understanding the Phase 4 iptables output. *Level: Advanced*

kube-proxy IPVS Mode [33]
https://kubernetes.io/docs/concepts/services-networking/service/#proxy-mode-ipvs
Describes the IPVS mode alternative to iptables. Relevant for Tier 2 learning. *Level: Intermediate*

12.8 CoreDNS and Kubernetes DNS

DNS for Services and Pods [29]
https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/
Official documentation of FQDN format, search domains, ndots:5, and headless service DNS. Explains the /etc/resolv.conf content seen in Phase 4. *Level: Beginner*

CoreDNS Manual [10]
https://coredns.io/manual/toc/
Official CoreDNS documentation including the kubernetes plugin, forward plugin, and Corefile configuration. *Level: Intermediate*

Debugging DNS Resolution in Kubernetes [28]
https://kubernetes.io/docs/tasks/administer-cluster/dns-debugging-resolution/
Official Kubernetes troubleshooting guide for DNS. Validates the Phase 4 triage approach with official methodology. *Level: Beginner*

12.9 NetworkPolicy

Network Policies [34]
https://kubernetes.io/docs/concepts/services-networking/network-policies/
Official documentation with clear examples of ingress/egress, podSelector, namespaceSelector, and ipBlock. Read before Phase 3. *Level: Beginner*

Network Policy Editor (Isovalent) [19]
https://networkpolicy.io/
Interactive visual editor for building NetworkPolicy YAML with an immediate graphical representation of what traffic is allowed. Excellent for validating policy logic before applying to a cluster. *Level: Beginner*

Kubernetes Network Policy Recipes [1]
https://github.com/ahmetb/kubernetes-network-policy-recipes

Curated collection of real-world NetworkPolicy patterns: default-deny, allow-within-namespace, deny-all-egress, allow-external-traffic. Extends the three-tier model from Phase 3. *Level: Intermediate*

12.10 Linux Namespaces, veth, iptables, and conntrack

Linux Network Namespaces man page [39]
https://man7.org/linux/man-pages/man7/network_namespaces.7.html
The authoritative reference for Linux network namespaces — the foundation of all pod network isolation in Kubernetes. *Level: Intermediate*

iptables Tutorial (Oskar Andreasson) [2]
https://www.frozentux.net/iptables-tutorial/iptables-tutorial.html
Comprehensive iptables reference including the nat table, PREROUTING DNAT, and chain traversal order. Explains every iptables concept in the kube-proxy rule structure. *Level: Intermediate*

Linux Conntrack: Design and Implementation [5]
https://arthurchiao.art/blog/conntrack-design-and-implementation/
Deep technical analysis of the conntrack subsystem, tracking entry structure, NAT state, and the exhaustion problem. The best single article for understanding the conntrack DNAT pairs seen in Phase 5. *Level: Advanced*

Netfilter Documentation [43]
https://www.netfilter.org/documentation/
Official Linux kernel packet filtering framework documentation. Covers iptables, nftables, conntrack, and the hook architecture. *Level: Advanced*

veth(4) man page [40]
https://man7.org/linux/man-pages/man4/veth.4.html
The kernel documentation for virtual ethernet devices. Explains the veth pair model and namespace assignment used by every CNI plugin. *Level: Intermediate*

12.11 tcpdump and Packet Capture in Containerised Environments

tcpdump man page [48]
https://www.tcpdump.org/manpages/tcpdump.1.html
Authoritative reference for all tcpdump flags, BPF filter expressions, and output format. Essential companion for Phase 5. *Level: Beginner*

nsenter man page [54]
https://man7.org/linux/man-pages/man1/nsenter.1.html
Authoritative reference for the nsenter tool used throughout Phase 5. Explains all namespace types: -n (network), -m (mount), -p (pid), -u (uts). *Level: Intermediate*

tcpdump Filters Cheat Sheet [46]
https://packetlife.net/media/library/12/tcpdump.pdf
One-page reference for BPF filter expressions. Covers host, port, net, proto

filters and their combinations. Useful for Phase 5 capture filtering. *Level: Beginner*

kubectl debug with Ephemeral Containers [27]
https://kubernetes.io/docs/tasks/debug/debug-application/debug-running-pod/
The officially supported alternative to nsenter for production clusters. Use an ephemeral container with nicolaka/netshoot to run tcpdump inside a running pod. *Level: Intermediate*

12.12 etcd

etcd Documentation v3.5 [13]
https://etcd.io/docs/v3.5/
Official documentation including clustering, operations, TLS, disaster recovery, and performance tuning. Phase 7 uses a v3.5.0 cluster. *Level: Beginner–Intermediate*

Operating etcd for Kubernetes [35]
https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/
Kubernetes-specific etcd operations: backup/restore, cluster health, compaction, and defragmentation. Covers the production best practices that this lab's no-PVC setup deliberately violates. *Level: Intermediate*

etcdctl Interacting v3 Guide [15]
https://etcd.io/docs/v3.5/dev-guide/interacting_v3/
All etcdctl commands used in Phase 7: endpoint health, endpoint status, put, get, watch, lease. *Level: Beginner*

etcd Raft Design [14]
https://etcd.io/docs/v3.5/learning/design-raft/
How etcd implements Raft, including the pre-vote extension, leader lease, and ReadIndex for linearisable reads. Advanced reading after Phase 7. *Level: Advanced*

12.13 Raft Consensus Algorithm

In Search of an Understandable Consensus Algorithm [45]
https://raft.github.io/raft.pdf
The original Raft paper by Ongaro and Ousterhout (2014). Readable and well-structured. Explains leader election, log replication, safety proofs, and membership changes. Essential reading after Phase 7. *Level: Intermediate*

The Secret Lives of Data — Raft Visualisation [21]
https://thesecretlivesofdata.com/raft/
Animated, step-by-step interactive visualisation of leader election and log replication. The best way to internalise Raft before reading the paper. *Level: Beginner*

Raft GitHub Pages [44]
https://raft.github.io/
The Raft project page, with the paper, slides, and links to implementations

in many languages. Also links to the TLA+ formal specification. *Level: Beginner–Advanced*

12.14 Goldpinger

Goldpinger GitHub Repository [4]
https://github.com/bloomberg/goldpinger
Official repository with Helm chart, Prometheus metrics documentation, Grafana dashboards, and RBAC manifests. Phase 6 uses the deployment pattern documented here. *Level: Beginner*

Bloomberg Engineering: Goldpinger [3]
https://www.techatbloomberg.com/blog/goldpinger-kubernetes-troubleshooter/
Bloomberg's original blog post explaining why they built Goldpinger, what failure modes it detects in production, and how they interpret the dashboard. Provides context for the Phase 6 observability goals. *Level: Beginner*

12.15 WSL2 Networking Caveats

Accessing Network Applications with WSL [41]
https://learn.microsoft.com/en-us/windows/wsl/networking
Official Microsoft documentation of WSL2 network topology, port forwarding behaviour, and known limitations. Explains the root cause of Issues 1–3 in this project. *Level: Beginner*

Docker Desktop on WSL2 Backend [11]
https://docs.docker.com/desktop/wsl/
Alternative Docker setup on WSL2 using Docker Desktop which integrates more reliably with the WSL2 network stack than a manual Docker CE installation. *Level: Beginner*

WSL2 Networking Deep Dive (Microsoft DevBlog) [42]
https://devblogs.microsoft.com/commandline/whats-new-for-wsl-in-windows-11-build
Technical deep-dive from the WSL2 team on the internal network topology. Explains the Hyper-V virtual switch, the NAT layer, and why Docker bridge routes are unreachable from WSL2 userspace. *Level: Intermediate*

12.16 eBPF and Cilium (Next-Generation Networking)

eBPF.io Learning [12]
https://ebpf.io/what-is-ebpf/
The authoritative introduction to eBPF: what it is, how it works in the kernel, and why it enables Cilium to replace kube-proxy entirely. *Level: Beginner*

Cilium Documentation [20]
https://docs.cilium.io/en/stable/
Official Cilium documentation. After understanding Calico (Phase 2), Cilium is the natural next CNI to study. Cilium uses eBPF instead of iptables, replacing both Calico and kube-proxy. *Level: Intermediate*

Life of a Packet in Cilium [7]
https://docs.cilium.io/en/stable/network/concepts/
Traces the path of a packet through the Cilium datapath using eBPF maps, tail calls, and socket-level load balancing. Directly comparable to the iptables-based path traced in Phases 4–5 of this lab. *Level: Advanced*

12.17 Quick Reference: Which Source for Which Question

Table 19: Rapid source lookup by question type

Question	Best Source
"Why is my node NotReady?"	kind Known Issues [23]; Calico architecture [49]
"How does kube-proxy DNAT work?"	Arthur Chiao's kube-proxy deep dive [6]
"What does conntrack store?"	Arthur Chiao's conntrack article [5]
"How do I write a NetworkPolicy?"	NetworkPolicy Editor [19]; Recipes [1]
"How does Raft work?"	Raft visualisation [21]; Raft paper [45]
"What does etcdctl output mean?"	etcdctl guide [15]
"Why does DNS use ndots:5?"	K8s DNS docs [29]
"How do I debug a pod network issue?"	kubectl debug [27]; tcpdump man [48]
"What does Calico IPAM do?"	Calico IPAM docs [50]
"Why is my WSL2 kubectl timing out?"	kind WSL2 docs [25]; WSL2 networking [41]

References

[1] Ahmet B. Alp. Kubernetes Network Policy Recipes. https://github.com/ahmetb/kubernetes-network-policy-recipes, 2022. Real-world NetworkPolicy patterns including default-deny and egress controls.

[2] Oskar Andreasson. iptables Tutorial 1.2.2. https://www.frozentux.net/iptables-tutorial/iptables-tutorial.html, 2012. Comprehensive iptables reference including nat table and DNAT.

[3] Bloomberg Engineering. Goldpinger: Kubernetes Troubleshooter. https://www.techatbloomberg.com/blog/goldpinger-kubernetes-troubleshooter/, 2019. Bloomberg's production use case for Goldpinger and failure detection.

[4] Bloomberg Engineering. Goldpinger: Kubernetes Pod-to-Pod Connectivity Monitor. https://github.com/bloomberg/goldpinger, 2024. Official Goldpinger repository: DaemonSet manifests, Prometheus metrics.

[5] Arthur Chiao. Conntrack: Design and Implementation in Linux Kernel. https://arthurchiao.art/blog/conntrack-design-and-implementation/, 2020. Deep analysis of conntrack entry structure and NAT state machine.

[6] Arthur Chiao. Cracking Kubernetes Node Proxy (aka kube-proxy). https://arthurchiao.art/blog/cracking-k8s-node-proxy/, 2022. Definitive deep-dive into kube-proxy iptables chain structure.

[7] Cilium Authors. Cilium Network Concepts. https://docs.cilium.io/en/stable/network/concepts/, 2024. Life of a packet through the Cilium eBPF datapath.

[8] CNI Authors. CNI Reference Plugins. https://github.com/containernetworking/plugins, 2024. Reference implementations of CNI plugins including bridge and host-local.

[9] CNI Authors. Container Network Interface (CNI) Specification. https://github.com/containernetworking/cni/blob/main/SPEC.md, 2024. The CNI ADD/DEL contract between kubelet and network plugins.

[10] CoreDNS Authors. CoreDNS Manual. https://coredns.io/manual/toc/, 2024. Official CoreDNS docs: kubernetes plugin, Corefile configuration.

[11] Docker, Inc. Docker Desktop WSL2 Backend. https://docs.docker.com/desktop/wsl/, 2024. Docker Desktop on WSL2: more reliable network integration than Docker CE.

[12] eBPF Authors. What is eBPF? https://ebpf.io/what-is-ebpf/, 2024. Authoritative introduction to eBPF in the Linux kernel.

[13] etcd Authors. etcd Documentation v3.5. https://etcd.io/docs/v3.5/, 2024. Official etcd v3.5 documentation: clustering, operations, TLS.

[14] etcd Authors. etcd Raft Design. https://etcd.io/docs/v3.5/learning/design-raft/, 2024. How etcd implements Raft: pre-vote, leader lease, ReadIndex.

[15] etcd Authors. Interacting with etcd. https://etcd.io/docs/v3.5/dev-guide/interacting_v3/, 2024. etcdctl v3 command reference: endpoint status, put, get, watch.

[16] Helm Authors. Helm Chart Best Practices. https://helm.sh/docs/chart_best_practices/, 2024. Production patterns for Helm chart design.

[17] Helm Authors. Helm Documentation. https://helm.sh/docs/, 2024. Official Helm documentation: charts, templates, releases.

[18] Kelsey Hightower. Kubernetes the Hard Way. https://github.com/kelseyhightower/kubernetes-the-hard-way, 2023. Manual cluster bootstrap guide exposing all internal components.

[19] Isovalent. Kubernetes Network Policy Editor. https://networkpolicy.io/, 2024. Interactive visual NetworkPolicy YAML editor.

[20] Isovalent / Cilium Authors. Cilium Documentation. https://docs.cilium.io/en/stable/, 2024. Official Cilium CNI documentation: eBPF-based networking and security.

[21] Ben Karau. The Secret Lives of Data — Raft. https://thesecretlivesofdata.com/raft/, 2014. Animated interactive visualisation of Raft leader election and replication.

[22] kind Authors. kind Configuration Reference. https://kind.sigs.k8s.io/docs/user/configuration/, 2024. All kind cluster configuration options including CNI and subnet settings.

[23] kind Authors. kind Known Issues. https://kind.sigs.k8s.io/docs/user/known-issues/, 2024. WSL2 and Docker networking limitations affecting kind clusters.

[24] kind Authors. kind Quick Start. https://kind.sigs.k8s.io/docs/user/quick-start/, 2024. Official kind cluster creation and configuration guide.

[25] kind Authors. Using kind with WSL2. https://kind.sigs.k8s.io/docs/user/using-wsl2/, 2024. Official kind WSL2 guidance and docker exec alias pattern.

[26] Kubernetes Authors. Cluster Networking — Kubernetes. https://kubernetes.io/docs/concepts/cluster-administration/networking/, 2024. Kubernetes network model requirements for CNI plugins.

[27] Kubernetes Authors. Debug Running Pods — Kubernetes. https://kubernetes.io/docs/tasks/debug/debug-application/debug-running-pod/, 2024. kubectl debug with ephemeral containers: production nsenter alternative.

[28] Kubernetes Authors. Debugging DNS Resolution. https://kubernetes.io/docs/tasks/administer-cluster/dns-debugging-resolution/, 2024. Official Kubernetes DNS troubleshooting procedure.

[29] Kubernetes Authors. DNS for Services and Pods — Kubernetes. https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/, 2024. FQDN format, search domains, ndots:5, and headless service DNS.

[30] Kubernetes Authors. kubectl Cheat Sheet. https://kubernetes.io/docs/reference/kubectl/cheatsheet/, 2024. Quick reference for all kubectl commands.

[31] Kubernetes Authors. kubectl Commands Reference. https://kubernetes.io/docs/reference/generated/kubectl/kubectl-commands, 2024. Complete kubectl subcommand reference with all flags.

[32] Kubernetes Authors. Kubernetes Concepts. https://kubernetes.io/docs/concepts/, 2024. Official Kubernetes conceptual documentation.

[33] Kubernetes Authors. Kubernetes Service Proxy: IPVS Mode. https://kubernetes.io/docs/concepts/services-networking/service/#proxy-mode-ipvs, 2024. IPVS mode kube-proxy: O(1) lookup alternative to iptables.

[34] Kubernetes Authors. Network Policies — Kubernetes. https://kubernetes.io/docs/concepts/services-networking/network-policies/, 2024. Official NetworkPolicy documentation with ingress/egress examples.

[35] Kubernetes Authors. Operating etcd Clusters for Kubernetes. https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/, 2024. Kubernetes-specific etcd operations including backup and restore.

[36] Kubernetes Authors. Organizing Cluster Access Using kubeconfig Files. https://kubernetes.io/docs/concepts/configuration/organize-cluster-access-kubeconfig/, 2024. kubeconfig structure and why stale config causes connectivity failures.

[37] Kubernetes Authors. Service — Kubernetes. https://kubernetes.io/docs/concepts/services-networking/service/, 2024. Official Services documentation: ClusterIP, headless, iptables mode.

[38] learnk8s.io. A Guide to the Kubernetes Networking Model. https://learnk8s.io/kubernetes-network-packets, 2023. Detailed packet-level walkthrough of pod networking with diagrams.

[39] Michael Kerrisk. network_namespaces(7) — Linux Manual Page. https://man7.org/linux/man-pages/man7/network_namespaces.7.html, 2024. Authoritative reference for Linux network namespaces.

[40] Michael Kerrisk. veth(4) — Linux Manual Page. https://man7.org/linux/man-pages/man4/veth.4.html, 2024. Kernel documentation for virtual ethernet device pairs.

[41] Microsoft Corporation. Accessing Network Applications with WSL. https://learn.microsoft.com/en-us/windows/wsl/networking, 2024. Official WSL2 networking documentation: port forwarding and limitations.

[42] Microsoft Windows Command Line Team. What's New for WSL in Windows 11. https://devblogs.microsoft.com/commandline/whats-new-for-wsl-in-windows-11-build-22000/, 2021. Technical deep-dive on WSL2 Hyper-V network topology.

[43] Netfilter Authors. The Netfilter Project. https://www.netfilter.org/documentation/, 2024. Linux kernel packet filtering framework documentation.

[44] Diego Ongaro. Raft Consensus Algorithm. https://raft.github.io/, 2024. Raft project page: paper, slides, implementations, TLA+ spec.

[45] Diego Ongaro and John Ousterhout. In Search of an Understandable Consensus Algorithm (Extended Version). https://raft.github.io/raft.pdf, 2014. Original Raft paper: leader election, log replication, safety proofs.

[46] PacketLife.net. tcpdump Filters Cheat Sheet. https://packetlife.net/media/library/12/tcpdump.pdf, 2018. One-page BPF filter expression quick reference.

[47] Kevin Sookocheff. Understanding Kubernetes Networking Model. https://sookocheff.com/post/kubernetes/understanding-kubernetes-networking-model/, 2022. Clear explanation of Kubernetes network model requirements.

[48] tcpdump/libpcap Authors. tcpdump(1) — Manual Page. https://www.tcpdump.org/manpages/tcpdump.1.html, 2024. Authoritative tcpdump flag and BPF filter reference.

[49] Tigera, Inc. Calico Architecture Overview. https://docs.tigera.io/calico/latest/reference/architecture/overview, 2024. Component architecture: calico-node, calico-typha, calico-kube-controllers.

[50] Tigera, Inc. Calico IP Address Management. https://docs.tigera.io/calico/latest/networking/ipam/, 2024. Per-node block allocation, blockSize, and IP pool management.

[51] Tigera, Inc. Calico Network Policy. https://docs.tigera.io/calico/latest/network-policy/, 2024. How Calico enforces Kubernetes NetworkPolicy via iptables and eBPF.

[52] Tigera, Inc. Calico Networking. https://docs.tigera.io/calico/latest/networking/, 2024. Official Calico networking documentation: BGP, VXLAN, IPAM.

[53] Tigera, Inc. Tigera Operator Installation. https://docs.tigera.io/calico/latest/operations/operator-based-installation, 2024. Operator-based Calico installation used in Phase 2.

[54] util-linux Authors. nsenter(1) — Linux Manual Page. https://man7.org/linux/man-pages/man1/nsenter.1.html, 2024. Authoritative nsenter reference: namespace type flags and usage.